

1. running todo list

1.1. Postmortem extraction plan (18 September 2026)

236. ☒ Extract cohesive leaf views with explicit imports: Explorer rendering,

popup/menu geometry and package browser. Delete original copies; record root and total production line changes. Preserve behavior without adding state or workers. Implemented in `src/app/explorer_view.rs`, `package_browser.rs` and `popup_layout.rs`. The original definitions are deleted. Relative to the working-tree snapshot before this extraction, `app.rs` is 14,033 → 12,840 lines (−1,193); `app.rs` plus these modules is 14,102 (+69 lines of module/import and formatting overhead). This is relocation, not a code-size or performance win. Existing test-only helpers move unchanged; the separate architecture boundary test adds 20 lines. Explorer ownership/effects in `show_workspace` remain for 238. No runtime scheduling, allocations, repaint policy or native bounds changed. Verification: all 28 focused Explorer tests, 1,058 full-suite test executions (16 opt-in tests ignored), 13 xtask tests, formatting and strict all-target Clippy pass. No screenshot or timing claim is needed for unchanged rendering and work; native navigation acceptance remains explicitly part of 237, not this extraction.

237. ☐ Consolidate Problems, index and preview navigation/focus handoff after

reproducing the native failure. Test document/range routing and secondary owners; do not equate helper tests with verified native word-navigation behavior.

238. ☐ Narrow Explorer ownership to explicit tree/index/selection inputs and

typed actions, with watcher effects outside painting. Remove full `EditorApp` access from the view and relocate its corresponding presentation state.

239. ☐ Thin save completion around existing receipts: consolidate active/parked

identity and continuations without weakening disk leases or durability. Cover same-path, Save As, close and stale results; require net-negative production code.

240. ☐ Inventory preview connection/content/visibility/generation writers and

remove duplicate policy from receive/restart adapters using existing controllers. Keep one policy owner, no extra retry state, per-frame copies or repaint loops. Work sequentially; report accounting and regression evidence per extraction.

1.2. Regression execution follow-up (18 September 2026)

228. ☒ Fix the no-document retained-host Settings panic at

`src/app/settings_view.rs:98`: preview unavailability does not imply a non-Typst document, because an empty tab store retains a Typst placeholder. Reproduced without manual input using `RUST_BACKTRACE=1 cargo xtask profile --scenario no-window --warmup 5 --seconds 5 --sampler none --skip-build on d3ca9886`. The actual hidden-host UI regression reproduced this panic before the fix and now covers hidden/visible Settings and resumption. Empty hosts report “No document”; New Window, Open in New Window and Settings remain available, while plain New/Open and document actions are disabled. Command-dispatch regressions cover reopening. Native close/reopen and the Open in New Window picker were exercised in an isolated app; file selection in that picker was not completed because the automation clipboard operation timed out. The native no-window runner also exposed an early-close CGL failure in its own transition: it now closes after the owner UI pass, registering the retained Settings surface just like a native close, and completes without panic. No polling or repaint loop was added. Evidence and remaining limitations: `regression-results.typ`.

229. ☐ Restore a reliable native hover profiling capture: the hover scenario

timed out before readiness after 120 seconds on the same optimized build while main, Settings and multi-window scenarios completed. Diagnose capture/event delivery before interpreting this as an application hang; preserve the timeout log and validate actual popup entry/scroll separately from helper tests.

1.3. Existing tasks

231. ☒ Use one app-wide Settings viewport owned by the retained process host.

Route secondary-window and native-menu requests there, raise it on repeated requests, retain it when hidden, and remove per-document toolbar toggle coloring. Keep preference broadcasts to all documents. Add secondary-owner, fixed-viewport and native-command routing regressions; native multi-window reopening exercised.

232. ☒ Let Open and Open in New Window select a file or folder. Folder selections

create an independent workspace with no open tabs and do not replace the current document. Explicit folder launches ignore last-file history. Test remembered-file suppression, empty tab state and no Tynymist start; inspect a native folder launch.

233. ☒ Do not restart or schedule the designated preview when Cmd+N adds an

editor tab. Preserve preview ownership and service generation, register only the new untitled LSP document with real backing, and keep its Git/editor state local. Add a generation/status/deadline regression; native Cmd+N preserved the existing WebView endpoint and rendered document.

234. ☐ Isolate the transient oddly shaped large-window flash reported when

opening Settings with multiple main windows. Duplicate Settings ownership is fixed in 231, but steady-state native observations/framebuffers cannot certify that a short-lived flash is gone. Keep this separate from singleton acceptance.

235. ☒ Take over manual checklist cases 4, 7, 8 and 9 with disposable-file and

deterministic stale-result tests. Record user passes for 1–3 without requiring another manual run. See regression-results.typ for exact test evidence and native-interaction limits; do not mark unobserved dialogs or timing races as native passes.

230. ☒ Identify executable builds in Settings → Status and `--version / -V`

using package version, Git revision, dirty state and a build-time Unix timestamp. Refresh metadata on source/assets/Git changes, handle builds without Git, and keep all identification work out of runtime frame paths. Deterministic QA captures use a fixed label. Add a CLI regression that exits before GUI startup and document clean builds versus separately installed app bundles in README.

Keep this list as the source of truth. Every task has a permanent number and ☐ (open, partial, unverified, or deferred) or ☒ (completed). Never remove completed tasks or renumber existing tasks; append new tasks with the next unused number. Resolved and audit notes must cite the relevant item numbers. Update the checkbox only when the entire task is complete and verified.

1. ☐ (Deferred) there should be a proper pdf preview option that is just a proper pdf viewer. maybe a webview?
2. ☒ Establish repeatable profiling builds, isolated representative workloads, CPU samples and bounded subsystem timings, plus an ongoing performance-review policy. (Resumed from deferred perf; individual optimizations remain ongoing work.)
3. ☒ typst overrides should be in appearances subsection

4. ☒ the color indicator in typst overrides should always be the current color, not faded out if its 'theme/override'. In fact there should not be a 'theme/override' button, since if we want to return to the theme we can just hit the reset button
5. ☒ somethings not right with alignment, see pic
6. ☒ lots of tooltips in typst overrides popup kinda suck. Too big, not useful. If useless just delete it, otherwise tighten the space
7. ☒ next to the theme name in the selectors, we should be able to see at a glance a colour palette of the theme.
8. ☒ allow customising UI fonts
9. ☒ cmd +/- should adjust UI size. Also put a slider for this in settings
10. ☒ syntax highlight the markdown in the tooltips. If typc doesnt give colour coding then treat it as typ.
11. ☒ if we are using a variable width font, add support to set the weight (typst overrides, and UI)
12. ☒ change so that: allow triangular area from cursor to tooltip / error window to not dismiss. When mouse is over the popup, we can scroll.
13. ☒ also add a keyboard shortcut to make the popup appear with keyboard focus (for scrolling up/down). Esc to unfocus and dismiss. There should be a subtle outline to indicate focus (as well as mouseover)
14. ☒ I noticed that when I use a ... code block, it is not syntax highlighted correctly. Fix
15. ☒ syntax highlight the markdown in the tooltips. If typc doesnt give colour coding then treat it as typ. (Duplicate of item 10; keep both statuses in sync.)
16. ☒ Mitex and cmarker integration: autodetect strings/raw strings inside `#mi(...)` etc to syntax highlight with tex or markdown. Note that you may need to put the text in math mode or text mode depending on the mitex command
17. ☒ for other colour formats (luma etc) we should give that text the appropriate fill, like we do now for `#rgb("hex number")`
18. ☒ package manager - inspect available packages, locally installed packages
19. ☒ detect when file changed outside app. If we dont have any changes queued to be saved, then reload the file
20. ☒ allow font ligatures in code editor
21. ☒ image / pdf preview on hover (both in code, when we include images as part of the code, and in file explorer)
22. ☒ configurable shortcuts everywhere. add a popup for this in settings
23. ☒ add search to all everything - explorer windows, settings,
24. ☒ view > ... needs to be in the title bar as well, and also none of the shortcuts are appearing - add
25. ☒ allow us to toggle off the title bar dupes of the menu bar items
26. ☐ (Deferred) once in a while the text everywhere breaks (see pic) some sort of leak?

27. ☒ after some time, the find/replace UI is not visible but is clickable. I realised this is because it is rendering behind the code editor panel. we shouldnt even make space for the find / replace UI - it should just be on top of the code panel.
28. ☒ keyboard shortcuts when find and replace is active should work on find and replace text fields. and similarly for other text fields
29. ☒ cmd+F / the Find button should toggle the find/replace popup, not just turn it on.
30. ☒ there should be a button to toggle find and find/replace
31. ☒ there should be options to allow case (in)sensitivity and regex (which we should be able to right click and get a cheat sheet). add single-symbol wide buttons for these
32. ☒ if we right click and its already 'use for preview'd then we should be able to toggle it off
33. ☒ text highlight of file names is not readable, adjust color scheme
34. ☒ should be able to right click file in explorer -> open in new window
35. ☒ cmd-/ to toggle comment out current or selected line(s)
36. ☒ shortcuts should be configurable (Duplicate of item 22; keep both statuses in sync.)
37. ☒ move the problems button to the rightmost slot (matching with default shortcuts)
38. ☒ remove 'cmd/ctrl' everywhere in popups/tooltips its just cmd
39. ☒ fallback for CJK fonts
40. ☒ autocomplete
41. ☒ text is selectable in too many places
42. ☒ file menu in menu bar does not match file in title bar. It should be single source of truth
43. ☒ file explorer subsection separators should be draggable
44. ☒ when closing the file explorer it should first stop showing all the contents before minimising
45. ☒ there should be another subsection in the explorer, the tags/references
46. ☒ color code various filetypes (various categories should be: .typ, other text-based files, pdfs and images, folders, and others) make sure you use colours from the theme
47. ☒ the ttt in settings window is no longer left of file name in main window(s)
48. ☒ the top right X in settings should not be there - use the Mac OS traffic lights
49. ☒ selecting a file in file explorer resets the file explorer scroll amount. Why? Opening a file cannot trigger more files to appear. It should not redraw. If it does, at least restore the scroll
50. ☒ the current line background should be also used for the current file. The currently opened file should also use an actual heavier UI-font face, not only stronger text color.
51. ☒ find and replace shortcut just gives find.
52. ☒ we should (by default) have the current scopes and the current sections/subsections etc be "sticky rows" at the top so that we have context for what we are looking at. each level should give a further row that is persisted to the top. For instance if we are in

```

= first section
// <many lines>
== subsection
#let a = {
// <many lines>
// <cursor is here>

```

then overlayed at the top should be

```

= first section
== subsection
#let a = {

```

(together with the line numbers). Basically another code panel floating on top but with those lines. There should also be line numbers, and clicking on the sticky row should jump in main code panel to that row

53. ☒ sync back from code -> preview should be one of the right click options
54. ☒ right click menu if possible should use native right click. If not, at least the text should not be centered, it should be left-aligned like normal
55. ☒ special detection of typst functions: set text(font: ...) should allow us to right click on font: ... and then have a scrollable selector
56. ☒ allow using fonts in the root of working directory or anywhere in the working directory if it is not a performance hit.
57. ☒ Table maker / editor on right click (with detection of tables)
58. ☒ the tooltips on hover should be wider. Maybe more like 80 char wide
59. ☒ remove unsafe rust as much as possible
60. ☒ it should format on manual save (cmd+s) (but not auto save)
61. ☒ if we have a file set to be used for preview, then when we switch to a different file, we dont need to restart the preview window. Also, we should not close it even if we open a non-typ file. thats the point of the used for preview setting - so that we can edit other child files like a .toml or other .typ files
62. ☒ Any part of the UI with this tiny font size like that used in the file path of the file explorer should have a bigger font. Never use this small font. Also don't use allcaps or smallcaps, not our style.
63. ☒ in addition to the pdf ready indicator i also want all the greentext on the side e.g. applied theme saved automatically. and everything should be timestamped.
64. ☒ we should reserve some space for the * in the name when a file is not yet saved, sot hat when it does appear, it doesn't shift the UI elements.
65. ☒ double clicking on a line in the Problems panel should jump to that line in the code panel
66. ☒ the logo icon should be redone to match the settings 'ttt'
67. ☒ on closing the app ran with cargo compile --release it should not lose the icon as it disappears?
68. ☒ Maintain UI observability and regression tests with targeted fresh screenshot inspection when pixel-level behavior is material. Capture detailed UI scenes in two representative themes (Catppuccin Latte and Mocha), and a generic main-window shot for each remaining theme.

69. ☒ code panel tooltips have disappeared. very odd?
70. ☒ if i am editing test.typ and i set todo.typ to be used for preview, then it should refresh the preview window to use todo.typ. this is not the current behavior, and if i then switch to opening the todo.typ file, the preview will still be stuck at test.typ
71. ☒ cmd+M should minimise the window, and so on for the usual mac os shortcuts
72. ☒ should have two proper dropdown for all fonts in settings (one for UI and one for code) - ability to choose any system font. In addition as mentioned above I think, we should search the local dir for fonts that we can use in the document.
73. ☒ the “Applied Catpuccin Latte” etc text is stuff i also want in the log. and keep the last 100 entries instead
74. ☒ font weight should not be adjusted until after mouse off the scrollbar as the UI shifts with the change causing feedback
75. ☒ remove the colour palette swatches from Settings completely
76. ☒ vertically centre the font-weight slider and give the code font its own independently persisted weight control
77. ☒ vertically align the Syntax label, live samples, and Reset buttons in Typst overrides
78. ☒ make popup focus and ownership robust so using or closing Settings cannot leave popups broken
79. ☒ keep the recent-status popup close to the status bar and right-align its timestamps
80. ☒ remove the obsolete pre-tiptotyp app bundle, registration, build outputs, and compatibility namespace completely
81. ☒ capture the complete maintained screenshot gallery from one app session instead of reopening the app for every image
82. ☒ redraw the refresh and used-for-preview eye vector icons so the refresh arrowhead does not overlap its body and the eye is rounded rather than angular
83. ☒ make tooltip Markdown links display only their linked text and open in the system browser; support opening external Typst `#Link(...)` targets with Cmd+click or the editor context menu; and open preview links in the browser
84. ☒ prevent selecting file-tree names, the ttt logo, and the title-bar filename; add file-name, absolute-path, and workspace-relative-path copy actions to file context menus; and show Cmd+1 through Cmd+5 shortcuts in the native View menu
85. ☒ add a Pause/Resume control for automatic preview updates and make Compile write the effective Typst entry’s PDF beside its source, using an output picker only for an unsaved document
86. ☒ fix the native startup deadlock that leaves the application unresponsive before any UI appears
87. ☒ allow dragging a file into the file explorer to add it into that folder. current file behavior should be scoped to falling on the code editor instead
88. ☒ we should be able to right click on a file and Show in Finder
89. ☒ when we reload from external inputs to the file, we should not need to reload the tinymist server since it was listening. I think just the code panel needs to be updated.
90. ☒ if we open a link in an external window then the greentext in the bottom should log that

91. ☒ I got this panic: `[?] cargo run --release` Finished release profile [optimized] target(s) in 0.55s
Running target/release/tiptotyp

thread 'main' (37498614) panicked at /Users/calvinkhor/.rustup/toolchains/stable-aarch64-apple-darwin/lib/rustlib/src/rust/library/core/src/num/f32.rs:1505:9: min > max, or either was NaN. min = 9.0, max = 8.0 note: run with RUST_BACKTRACE=1 environment variable to display a backtrace I was using two different windows with two different workspaces. Possibly i was choosing a file. I guess we need to make sure all the stuff between windows are completely separate

92. ☒ the file preview should appear on the right of the cursor and in particular not on top of the file tree so it is easy to browse

93. ☒ there should be a keyboard shortcut (or two?) to open/close the tooltip under the mouse cursor and under the keyboard cursor

94. ☒ We need to also be able to delete files from the explorer

95. ☒ Git integration - allow the most basic git functionality, enough for basic use.

96. ☒ when we are editing a non-typ file, grey out Code Split Preview buttons

97. ☒ completions should filter as i type more chars.

98. ☒ font completions should similarly filter (all filtering should be fuzzy match), I should be able to type `#set text(font: "` and then have the full list. current behavior is the completions show when I type `#set text(font:` but disappear on the space.

99. ☒ show a preview rendered in the selected font, both in font completion and font selection in Settings (clarified 2026-09-12)

100. ☒ there should be an uninstall button in the package UI

101. ☒ we should be able to access packages UI from View title bar

102. ☒ we should be able to rename from title bar File > Rename

103. ☒ there should be a button to go to the package desc on the website

104. ☒ Currently, we need to wait for the online packages to be fetched before any is shown. The installed packages should immediately appear, followed by the online generated list once they are ready.

105. ☒ when we comment a line out with `cmd-/` the ‘

106. ☒ There should be better completions for references. Firstly, the thing on the left should be the code, and after that should follow the other data. Secondly, it should filter as I type.

107. ☒ find and replace should appear on top of the sticky rows, not push them down

108. ☒ the image/pdf preview should not have so much text. just the image alone is enough.

109. ☒ the title-bar options (file, edit, view) do not mirror the title bar ones. make it so that it is a error, type error or other strict testing behavior that these two must always match. use a single source of truth.

110. ☒ browsing font completion and font-picker previews must not rebuild the application fonts or flash text throughout the UI

111. ☒ polish the Git window with right-aligned buttons, a visible diff viewer, and Unstage all

- 112. ☒ show Git status in Explorer and clickable change markers beside line numbers, with independent state and diff windows for every document window
- 113. ☒ audit correctness and maintainability across document workflows, file operations, Git, and background work; fix verified defects and redundant scans
- 114. ☐ crash recovery: restore unsaved buffers and all document windows after an unexpected exit
- 115. ☒ Git chunk actions: stage, unstage, and revert individual chunks, with buttons in the existing hunk popup and configurable keyboard shortcuts for actions and previous/next change. Reject stale selections and preserve unrelated staged/unsaved changes; revert through the editor's undo history. Implemented and regression-tested; see docs/tabs-and-git-hunks.md for controls, safety rules, visual evidence, and profiling results.
- 116. ☐ workspace search and replace: preview replacements across files, support regex capture groups, and provide undo
- 117. ☐ large-project performance: share repository status between windows and measure indexing, preview, and typing latency (ongoing optimization, using the profiling foundation in item 2)
- 118. ☐ PDF polish: document search, thumbnails, and reliable position restoration (extends deferred item 1)
- 119. ☒ extract settings, editor rendering, and window orchestration into focused modules with enforced ownership boundaries
- 120. ☒ implement refactor.typ R1–R8: sealed document mutations, workflow states, owned tasks, preview provenance, headless core, coordinate types, presentation ownership, and explicit write outcomes
- 121. ☒ The image preview in the file explorer appears to the right of the file name position. But this is not exactly right - it should appear to the right of the file explorer panel. This is different if the name is so long that we have to scroll the panel to see it. As currently it uses the file name length, this is past the panel boundary.
- 122. ☒ the currently selected file should not have a bigger font size.
- 123. ☒ it shouldn't say pdf is ready in X ms, it should say preview is ready
- 124. ☒ on first open, it says pdf is ready in 0 ms, even when it is not true. fix
- 125. ☒ when searching with find/find and replace, it should on each keystroke automatically jump in the code panel to the first match. currently it only jumps when we hit enter.
- 126. ☒ when searching with find/find and replace, all matches should immediately receive a highlight. currently it only has a highlight on the current line. that's good, but we also need a different highlight color for the matches.
- 127. ☒ popups in a main window stop working if we have another window open e.g. packages or settings window. the popups should either always work or at least work for the currently focused window.
- 128. ☒ close diff button is superfluous, remove it.
- 129. ☒ in fact let's not have a separate git hunk diff window at all. Have the diff show in a popup.
- 130. ☒ the git window should instead be a subpanel in the explorer window.
- 131. ☒ the tags and references subpanel should be two separate panels

132. ☒ allow us to choose the order of the explorer panels in Settings. git panel should default to under files
133. ☒ implement tabs in the existing title-bar filename area, each with a close × and adjacent sibling tabs. Preserve each buffer, dirty state, undo, cursor, and scroll position. Use the first tab as the preview entry automatically, with an explicit control to choose another; share window services instead of running a compiler/Tinymist instance per tab. Confirm dirty tab/window/process closes. Implemented with independent tab state, guarded background auto-save, preview-source circles, and distinct tab/window shortcuts; deterministic tests, fresh light/dark captures, and matched idle profiling are recorded in docs/tabs-and-git-hunks.md.
134. ☒ we should be able to click on the ttt logo in a window and have it open a color picker for the bg of the logo, which will help us visually identify each window.
135. ☒ when cursor is at a bracket/dollar sign/etc we should highlight the matching char.
136. ☒ when we type one of these bracket chars, we should by default automatically insert the matching char and place the cursor in between. If we backspace, from this, delete both. This should be configurable in settings.
137. ☒ we should also implement rainbow brackets. Each type of bracket pair ([], (), {}, and mixed brackets [], {}, etc) should use a different cycle of colors. Allow us to choose palletes to cycle through for the brackets in settings.
138. ☒ i noticed that if a popup appears because i hovered over something, then scroll, the scrolling is not performant, losing frames. Investigate and fix.
139. ☒ too much space is reserved for the git hunk color marker on the side. It should take at most 1 char width or other similar small measurement.
140. ☐ Supply missing Unicode glyphs in symbol completions, including Hebrew letters, mathematical operators, and alchemical symbols, without resetting fonts during interaction.
141. ☒ If the Tinymist server times out or fails, restart it automatically and use the raster fallback only after five consecutive failures.
142. ☒ Compress the Settings window for variable-width layouts, including compact bracket-family labels such as () instead of Parentheses ().
143. ☒ Keep the auto-save delay label and slider together when wrapping; wrap complete toolchain status chips onto the next line when space runs out.
144. ☒ Remove the “both themes · invert, then hue” text and allow any light or dark theme in either appearance slot, listing matching themes before opposite-mode themes.
145. ☒ Add persisted theme color adjustments for luminosity, brightness, contrast, and saturation, with reset controls and consistent application to UI and syntax colors.
146. ☒ Make Settings tooltips fit their content, keep useful explanations and hidden details, and remove hints that only repeat the visible control or status.
147. ☒ Combine inversion and hue shift with the other color adjustments in one responsive section with a single reset for every adjustment.
148. ☒ Make Tinymist recovery a clock-driven state machine: wait before retrying, fall back on the fifth consecutive failure, include preview-start failures, and test full recovery/event sequences.

149. ☒ Establish focused ownership for preview recovery, Explorer, Settings UI, and tooltip behavior instead of letting every view mutate unrelated EditorApp state.
150. ☒ Isolate screenshot fixtures behind a QA entry point and remove screenshot-only product UI; capture the same components and layouts used by normal windows.
151. ☒ Report malformed saved settings distinctly from missing settings, retain the rejected data for diagnosis, and test the recovery path.
152. ☒ Add automatic formatting, strict Clippy, application/core, and xtask checks, plus a separate real-tool integration job on a supported platform.
153. ☒ Investigate and reduce Settings-open CPU usage: eliminate redundant native-window updates/repaints and unchanged preference actions, preserve theme changes and reopen behavior, and verify with regression tests and before/after profiling.
154. ☒ Make shared hover popups cheap: dismiss on owner scrolling and pointer movement away, preserve movement into and scrolling inside the popup, delay unnecessary server requests, separate native repainting, avoid repeated parsing/cache copies, and remove fading. Compare full-document rendering with a short preview; reveal full content automatically on pointer entry or keyboard focus, without Read more buttons. Verify performance and interaction regressions.
155. ☒ Fix missing semantic hovers caused by context-free token targeting: honor math operator boundaries without splitting valid code identifiers, handle the right half of a final glyph, and reuse cached syntax/target queries. Cover multiple names and contexts, not symbol-specific exceptions.
156. ☒ Fix remaining Settings interaction latency: separate native Settings repainting from the editor, preserve preference/action synchronization and font/theme updates, stop animating previews that are waiting for window focus, and verify independent scrolling/idle behavior with regression tests and comparable optimized profiles.
157. ☒ Collapse the multiline constructs shared with sticky rows using a clickable line number or gutter triangle. Double the initial triangle size without widening the margin, keeping it between Git markers and numbers. Show clickable “...” at the end of each collapsed header, including wrapped headers. Preserve source offsets, copying, nested folds and unaffected edits; reveal explicit destinations and skip hidden rows during vertical navigation. Cache folded layouts and verify behavior, geometry and performance.
158. ☒ Share folding and sticky contexts for Markdown and TeX code blocks/strings (cmarker and MiTeX, including named arguments and tagged raw blocks). Use a source-offset-aware CommonMark parser and a lightweight TeX structural reader; honor sections, blocks/environments, comments and verbatim exclusions, and map escaped strings back to physical source lines. Cover Unicode and malformed/incomplete input without adding another LSP. Implementation and measurements: docs/editor-folding.md.
159. ☒ Audit multi-window ownership and the no-window state. Fix stale native-command focus, hidden-root selection after close, Dock reopen, Finder-open wakeups, discarded root buffers/autosave, concurrent Settings edits and completion notice routing. Preserve no-window New/Open/Settings and child clipboard commands. Verify deterministic ownership/state regressions; findings and native-QA limitations: docs/multi-window-audit.md.
160. ☒ Give the retained hidden root a dormant-host lifecycle. Stop document rendering, watcher/LSP sessions and recurring workspace/index work; retain Settings, native commands, dialogs and operation completions through the hidden-window logic hook. Resume services once after queued replacements, retain a hidden Settings viewport, and block idle workers instead of polling. Preserve

that native Settings surface across reopening to avoid the macOS CGL panic; disable New and document/edit commands when no document is open, and refresh menus in hidden logic. Verify lifecycle/routing regressions and matching optimized no-window profiles; evidence and native-QA limits: docs/multi-window-audit.md.

- 161. ☒ Keep editing and jump-to-source coordinates correct with collapsed lines. Remap unaffected folds inside TextEdit's mutation-time layouter, and install explicit destinations before the old caret can reveal a fold or trigger scrolling. Regression coverage checks Unicode offsets, edit-frame scrolling, and preview-style source jumps.
- 162. ☒ Eliminate the expanded-line flash while typing with folds. Every edit-time galley now preserves unaffected folds; only touched regions expand. Test the actual input frame and its agreement with the committed revision, including line insertion/deletion and Unicode edits.
- 163. ☐ Allow for the pdf preview to pop out into a different window. if I close it, the preview should go back to the window
- 164. ☒ Restore native shadows for newly created document windows (the initial/opened window was unaffected). Give secondary document viewports opaque backing and explicitly enable their native shadow; preserve root and popup transparency behavior. Builder regressions and a native New Window smoke check pass; exterior-shadow appearance still needs manual confirmation because the available capture crops at the window boundary. See docs/multi-window-audit.md.
- 165. ☒ Restore visible "Preview ready in X ms" feedback, including interactive Tinymist compile cycles. Add successful New/Open/Save/Auto-save notices to the existing bounded status history. Show the latest action or preview message in one shared location left of the status icon, retaining previous messages in history. Compact Git counts to +/- notation. Ignore duplicate completion reports and unrelated entries/generations; do not invent startup timing. See docs/status-feedback.md.
- 166. ☒ Center the spinner and "Building preview..." as one group, including wrapped messages; preserve static focus-wait behavior. Geometry regressions and a fresh viewport inspection pass.
- 167. ☒ Left-align Explorer subpanel titles while retaining their full-width click targets. Semantic click and painted-position regressions pass; fresh viewport inspected. Full gallery refresh encountered a macOS OpenGL panic and restored its previous images; details in docs/status-feedback.md.
- 168. ☒ Add local TeX command completion in tagged raw blocks and MiTeX literals, with math/text-aware vocabulary and document-defined macros. Preserve Typst string escapes and Unicode edit offsets; exclude comments/verbatim and use the existing popup without another server. Cache the syntax-derived index per document revision. Tests, supported declaration forms, performance evidence and lexical-parser limits: docs/tex-completion.md.
- 169. ☒ Add optional TeX dollar notation with pinned MiTeX version and a selected miTeX toolbar control. Save inline x as `mi(...)` and Typst-style display x as `mitex(...)`, inserting the required imports. Refuse native Typst equations on enable; preserve exact existing call spelling, dirty state, saves and undo boundaries. Provide TeX highlighting/completion inside dollar pairs, mapped compiler/Tinymist edits and navigation, canonical Git diffs with mapped gutters, and canonical index/file-link navigation. Ordinary mode bypasses translation, map/index construction and canonical save-cache allocation; deterministic regressions assert zero projection work, and matched optimized measurements show no consistent disabled-mode penalty. Real Typst and Tinymist compile both forms successfully. Behavior, limitations, performance evidence and UI validation: docs/mitex-projection.md.
- 170. ☐ use libghostty and get a terminal window open. it should be either the problems panel or a terminal.

171. ☒ Keep TeX math commands such as alpha available in text-classified TeX blocks; use math/text context to rank equally matching suggestions instead of removing them. Cover the final filtered alpha-prefix results and contextual preference ordering.
172. ☒ With automatic delimiter pairing enabled, Enter after an opening triple-backtick fence and optional language inserts its closing fence, including inside MiTeX calls. Enter inside an empty Typst math pair creates a body line at the existing indentation plus two spaces, then a separate closing dollar. Preserve caret, atomic undo/redo, existing closers, literal exclusions, paste/IME and disabled-setting behavior; block-opening Enter takes precedence over completion acceptance. Ordinary Enter does not parse. See docs/tex-completion.md. Follow-up: a Space typed inside an auto-created empty \$. . . \$ pair inserts matching spaces on both sides of the caret, and fenced-block completion remains independent of later tagged raw-block openers.
173. ☒ Reorder the title bar: traffic lights, ttt, File Edit View | file title | right-aligned miTeX Find Pause Compile | Settings Explorer Code Split Preview Problems. Show miTeX only for compatible documents; keep active mode reachable during unfinished edits. Add a separate auto-enable preference for compatible documents, preserving manual toggles and opening incompatible files normally. Cache eligibility by source revision/package and reuse editor syntax. Verified narrow/wide semantic layouts, state transitions, optimized compatibility cost, fresh light/dark and Settings framebuffers, all 68 gallery images and required checks. Measurement and evidence: docs/mitex-projection.md.
174. ☒ Fix multi-document sluggishness: replace coupled immediate document viewports with independent deferred repainting while keeping native UI thread-affine. Preserve owner-specific menus/settings/close/native-parent routing; queue shared settings for owner-local application and prevent late callbacks/history resurrection. Add isolation regressions and a reusable four-window profiling scenario. Matched eight-second idle runs reduced total editor UI work from 380 ms to 66–108 ms, with process CPU advance 0.97 s versus 0.21–0.35 s; not an end-to-end FPS claim. Required ordinary/profiling checks pass. Evidence and limitations: docs/multi-window-audit.md.
175. ☐ ‘starter screen’ if we cannot find a working directory
176. ☒ Expand application/document/workspace keyboard coverage to 76 configurable actions, all with collision-free platform defaults: miTeX, preview-source selection, folding, Find/Replace actions, raster page/fit controls, Explorer search/refresh, status history, display toggles, menus and shortcut-editor access. Fill previously unassigned Rename/Packages/Git/Sync/Fullscreen defaults. Preserve effective overrides, most-specific routing, focus/file-operation guards and safe replacement. Individual Settings options and per-item dialog actions do not each get a global chord. Details: docs/keyboard-shortcuts.md.
177. ☒ Line numbers use the configured editor font and align actual glyph baselines, including sticky headers and taller fallback-font rows. Keep a three-point text gap and separate fold/Git lanes; size the column correctly for proportional digits. Geometry, folded-row and semantic tests pass; fresh light/dark framebuffers inspected. Evidence: docs/tabs-and-git-hunks.md.
178. ☒ Closing the last tab leaves an empty workspace window with Explorer and New/Open controls. PDF/image tabs use the editor pane while a pinned Typst preview remains visible, with independent asset loading, page navigation and zoom. Empty-workspace commands/reopening, stale results, no empty-document service work and split-pane/page-margin geometry have regression coverage. Required checks pass; fresh light/dark captures and the regenerated 68-image gallery were inspected. Matched local idle profiles showed no added work, not an interaction speedup. Evidence and limitations: docs/tabs-and-git-hunks.md.

- 179. ☒ Extend the selected-tab highlight across the title and both controls. Reuse the Explorer's vector eye with a filled pupil for the preview tab, add a centered closed-eye variant for other tabs, and draw a vertically aligned path-based close icon. Geometry and named-control regressions pass; fresh light/dark captures inspected and the 68-image gallery regenerated/validated. Evidence: docs/tabs-and-git-hunks.md.
- 180. ☒ Cmd+N opens a new tab without changing the designated preview tab or its source. Stable-ID regression coverage exercises repeated New operations with a different pinned preview.
- 181. ☐ (deferred) we should be able to open from template.
- 182. ☐ pretty animation for dragging tabs
- 183. ☐ prettier git diff

1.4. Architecture audit follow-up

Source: output/pdf/architecture-audit.typ, proposals A01–A12. These are implementation tasks, not completed audit findings. Each item should ship as a focused change, preserving existing behavior unless explicitly stated otherwise. Review corrections (2026-09-17): prioritize correctness fixes 213 and 207, then small boundary extractions such as 202. Establish 184 and 187 before changing document storage or save timing. Preview and background-work tracks can proceed separately; neither requires a whole-application rewrite. Keep the existing core, native thread affinity, plain-mode projection bypass and protected-mutation completion guarantees. Record matched optimized before/after measurements for performance-sensitive changes; use deterministic invariants rather than timing thresholds. These tasks refine item 117, not replace its broader multi-window measurement and shared repository-status goals. PDF resource work does not include the viewer features in items 1, 118 or 163.

- 184. ☒ (A01) Centralize document access through the existing stable tab IDs before changing storage; do not invent a second identity system. Reuse existing tests and add missing characterization coverage for reorder, active/preview identity, empty workspace, dirty close, and switching with retained undo/selection; preserve the preview-selection rule in item 180.
- 185. ☒ (A01; after 184) Replace the active hole and parallel tab vectors with a document store keyed by stable IDs, a separate order list, and optional active/preview IDs. Route existing callers through the accessors and remove obsolete storage; identity tests must pass unchanged.
- 186. ☒ (A01; after 185) Unify active and parked ownership of folding, saved editor-widget state and autosave metadata in the document records, reusing the fields already present in ParkedTab. Test stale replies across switches and that tab changes start no extra services; measure switch allocations and many-tab idle work. Keep services window-owned.
- 187. ☒ (A02) Add adversarial save-order tests before changing execution: edits during save, overlapping same-path requests, failed/uncertain writes, and close during completion. Assert that only the matching durable receipt can approve a close.
- 188. ☒ (A02; after 187) Wrap the existing core SaveRequest/SaveReceipt and durability outcomes in immutable worker inputs/results carrying canonical bytes, expected disk state, intent and continuation token. Do not duplicate core document identity or receipt validation. Adapt the existing save path without changing execution timing; test receipt identity and durability handling.
- 189. ☒ (A02; after 188) Execute save disk work as protected background transactions. Recheck expected disk state inside the app-owned path lease, serialize same-path writes, and retain completion after owner closure. A controllable slow-writer test must leave UI dispatch non-blocking; do not claim atomicity against external writers.

190.  (A02; after 189) Route active manual save and parked-tab autosave through the same coordinator and remove duplicated write policy. Preserve manual-only formatting and reject stale receipts; test save/close continuations for both paths without automatic conflict merging.
191.  (A03; after 184) Define versioned synchronization inputs and typed open/change/close/backing-update effects. Add command-log tests for edit, switch, close, preview-entry change and miTeX-mode change using current behavior as the contract.
192.  (A03; after 191) Extract canonical snapshot collection, open-URI tracking, private backing ownership and active/preview root selection into one synchronization coordinator. Keep filesystem and service IO in adapters; remove duplicate orchestration paths.
193.  (A03; after 192) Enforce document/generation checks at synchronization-result boundaries. Test rejection of late replies, canonical/display coordinate separation, and zero translation work for ordinary documents; explicitly retain the CLI imported-subfiles-from-disk limitation.
194.  (A04) Extract app-side preview transitions into event/effect functions over the existing connection, recovery and content models. Do not add parallel state or a second retry policy. Reuse existing tests and cover missing sequences for fifth-failure fallback, duplicate failures, late readiness, pause/restart, export during recovery and preview-entry changes.
195.  (A04; after 194) Route service start/stop, rendering and repaint requests through preview effects. Assert one effect per transition, no duplicate compilation and no idle repaint loop; keep native views and texture handles in adapters.
196.  (A04; after 195) Make preview rendering consume a read-only status snapshot distinguishing requested/effective backend, canonical artifact availability and native readiness. Remove view-side policy changes and test recovery/status agreement.
197.  (A05) Extract PDF rasterization and link extraction from the compiler into a reusable PDF service for compiled documents and opened assets. Preserve canonical export bytes and existing page/link results with focused parity tests.
198.  (A05; after 197) Separate all-page dimensions/metadata from decoded pixels and uploaded textures. Key resident pages by artifact, page, scale and relevant appearance revision; test stale artifact/theme rejection and retention of valid old content during replacement.
199.  (A05; after 198) Request raster pages by visible range with bounded adjacent-page prefetch and cooperative cancellation. Test scroll/zoom and replacement ordering so obsolete requests cannot replace current pages or discard valid displayed content prematurely.
200.  (A05; after 199) Enforce process-wide decoded-pixel and texture byte budgets with eviction that coordinates visible-page demand across windows. Add 1/20/100-page bounded-residency tests and matched optimized cold/warm, idle/scroll/zoom measurements; keep export quality unchanged and document oversized-page handling.
201.  (A06) Extract bounded LSP framing into a private transport module without changing sidecar commands/events or threads. Test malformed, oversized and truncated frames plus shutdown ordering; introduce no extra payload copies or queues.
202.  (A06; after 201) Move JSON-RPC feature DTOs/codecs into a UI-independent protocol module. Preserve existing wire fixtures and fake/real-server coverage; keep the public sidecar interface unchanged.
203.  (A06; after 202) Reassess the remaining session/process coupling after codec extraction. Extract only transition logic whose ownership becomes clearer; retain existing generation/document-

version fields and admission checks rather than duplicating them. Test stale replies and restart/shutdown ordering; do not introduce a generic LSP framework or move threads for organizational reasons.

204.  (A07) First reproduce superseded project-index jobs continuing to consume work and record queue/payload/concurrency bounds. Implement the smallest shared bounded read-job runner needed for that client: fixed concurrency, keyed pending replacement and byte accounting. Do not build a generic executor or priority hierarchy without a second demonstrated use case; test fair admission and keep protected mutations/long-lived protocol supervisors separate.
205.  (A07; after 204) Add cooperative cancellation checkpoints and owner/source-key completion routing to the executor. Test that superseding 100 requests cannot run 100 obsolete jobs and that closed owners receive no UI result; wake only the owning viewport.
206.  (A07; after 205) Migrate project indexing as the first executor client and remove its superseded per-request spawning path. Measure foreground latency under repeated indexing and multi-window load; retain literal-only analysis and bounded cancellation checkpoints.
207.  (A08) Extend ProjectIndex with completeness metadata for traversal-cap hits and unreadable local files; retain its existing unresolved-expression records rather than adding a duplicate dependency model. Surface concise warnings instead of implying complete results; test each case without evaluating Typst or indexing downloaded packages.
208.  (A08) Share immutable filesystem snapshots through a canonical-root workspace service while keeping each window's selection, expansion and search local. Test that two windows reuse an unchanged scan and closing one does not invalidate the other's snapshot; use 204–206 if adopting that executor.
209.  (A08; after 208) Coalesce filesystem notifications and use metadata filtering plus a conservative verification fallback instead of routine whole-file checks. Test atomic-save, delete, rename and missed-event recovery; measure scan/read counts across two windows and retain separate filesystem, language-index and Git models.
210.  (A09) Introduce a typed completion-edit transaction carrying source version, validated coordinate ranges, selection and one undo intent. Reuse core validation and miTeX preflight; test Unicode, CRLF, invalid ranges, stale replies and one-step undo while leaving typing/IME with TextEdit.
211.  (A09; after 210) Extract the completion popup renderer behind read-only inputs and typed actions, without access to EditorApp or services. Add semantic acceptance/selection tests and check unchanged-frame allocations; do not add full-source copies or broaden this patch to every editor feature.
212.  (A10) Extract Git command execution and status/diff codecs behind a repository service handle. Preserve argument-array invocation and add disposable-repository coverage for quoted paths, new files, CRLF and missing final newlines; this is not the visual redesign in item 183.
213.  (A10; independent of 212) Route hunk index mutations through the existing repository-root transaction lease and revalidate baselines under that lease. Share the transaction entry point with panel operations; do not postpone this correctness fix for a service extraction. Test overlap with panel operations, conflicts, partial staging and preservation of unrelated staged changes; retain protected completions.
214.  (A10; after 212 and 213) Make Git panel/gutter/popup views consume immutable state and emit typed actions only, removing service-to-UI dependencies. Test that rendering cannot execute Git

and that checked hunk revert remains one editor undo; preserve item 115's controls and safety rules.

- 215. ☒ (A11) Give child-view and popup resources explicit owner lifecycle hooks distinguishing temporary hide, durable close and dormant native hosting. Test reopen, scroll dismissal, keyboard focus and inert late callbacks without destroying every hidden view.
- 216. ☒ (A11; after 215) Dispose closed-viewport font-sample slots and other owner-keyed context caches through those hooks. Add repeated open/preview/close tests asserting live cache counts return to baseline or a documented bound; measure retained resources after repeated cycles.
- 217. ☒ (A11; deferred pending measurement, after 215) First trace redundant native property updates and measure their cost. Implement diffing only if it materially reduces work; invalidate applied-state caches on native recreation and external geometry changes. Preserve clipping/reopen behavior with geometry tests and native bounds traces; use proportionate whole-window observation for composition, not a viewport PNG. If no useful saving is found, record that result rather than adding a cache.
- 218. ☒ (A12) Move ordinary LaunchOptions parsing out of screenshot ownership into a launch module with explicit normal/capture/profile modes. Test existing argument behavior and QA/profile non-persistence; preserve tool-selection precedence.
- 219. ☒ (A12) Derive a cached capability snapshot from existing tool resolution and service state, separating editing/LSP, interactive preview, PDF generation, rasterization and link extraction. Expose missing raster/link capabilities separately; invalidate on tool-preference changes and explicit refresh, and test partial availability. Do not add a competing discovery/status service or probe tools in a frame callback.
- 220. ☒ (A12) Generate or validate runtime tool-version metadata from toolchain/manifest.tsv so constants and packaging cannot drift. Add a mismatch regression/build check; preserve resolution precedence without bundling new tools or retaining compatibility aliases.
- 221. ☒ (A12) Correct obsolete miTeX comments in src/lib.rs and add superseding current-state notes to ADR 0002 for per-window Tinymist, the separate Settings viewport and conditional CLI compilation. Cross-link the current ownership/module map without erasing historical decision context.
- 222. ☐ refactor to allow for a tex engine
- 223. ☐ prove out typst-compatible binaries like calepin
- 224. ☒ Audit the completed architecture refactors at their ownership and asynchronous-result boundaries. Fix rejected-work admission destroying valid jobs, stale workspace/Tinymist results, PDF process/page/residency errors, parked Save As rebinding, incomplete capability reporting and malformed Git-hunk arithmetic. Add focused regressions and retain bounded, event-driven resource use.
- 225. ☒ Restore reliable interactive tooltip handoff and preview-to-source keyboard focus. Use a fixed safe triangle from the opening cursor position to the popup's complete facing edge, never classify motion inside it as moving away, and do not restart hover delay because of a slow frame. After a Tinymist source jump, return native focus from the preview child to the editor so macOS Option+Left/Right word navigation works.
- 226. ☒ Correct the incomplete fixes in 225: keep retained tooltip overlays alive when the source hover disappears, prevent unrelated controls from clearing shared hover timers, restore the native first

responder with `WebView focus_parent`, and reuse revision-cached syntax for asset hover instead of parsing the document on every pointer frame. Add lifecycle, timer-ownership and syntax-reuse regressions.

227.  Repair undersized persisted Explorer widths on startup: restore the 230-point default when below the 160-point usable minimum, preserve valid saved widths and normal reopen behavior. Move partial project-index details into the workspace-header tooltip instead of showing a persistent warning row. The details describe potentially missing Explorer entries, not compilation failures. Add startup-size and warning-visibility regressions; no background jobs or repaint loops added.

2. Bounded PDF page residency (2026-09-18)

- Items 198–200: PDF inspection now publishes a page catalog containing only 144-DPI layout dimensions and normalized links. Decoded RGBA and egui texture handles live only in resident page slots keyed by artifact generation, page, requested DPI and appearance revision. A replacement artifact keeps the old catalog and pixels visible until the first valid replacement page arrives; late artifact and pre-theme-change replies are rejected.
- The fallback preview derives the intersecting page range from its scroll viewport, adds one adjacent page on each side and caps one request at 12 pages. Two latest-wins workers allow the designated Typst preview and an opened PDF pane to remain visible together. A changed scroll, zoom, artifact or appearance token cancels Poppler cooperatively; offscreen pages no longer construct image widgets. Metadata still provides stable whole-document geometry and page navigation before pixels arrive.
- Decoded pixels and estimated GPU texture bytes share process-wide accounting across all windows: 96 MiB decoded and 192 MiB texture budgets. Eviction prefers the least-recently-visible nonvisible page, wakes its owning viewport and drops the actual texture there. A single page larger than either budget is admitted alone after evicting other pages; this makes oversized pages usable without allowing multiple oversized residents to accumulate.
- Canonical PDF bytes remain in `PreviewContent` and are still the sole export source, so raster scale, appearance and eviction cannot change export quality. Deterministic tests cover metadata/rotation parsing, visible ranges, bounded prefetch, cooperative supersession, old-content retention, artifact/theme rejection, cross-window visible-page preference, oversized pages and bounded 1/20/100-page residency. The architecture guard prevents the compiler from regaining decoded-page ownership.
- The opt-in optimized resource-model probe ran on Apple M2 Max, arm64 macOS 14.6.1, Rust 1.96.0: `cargo test --release optimized_pdf_residency_probe -- --ignored --nocapture`. With synthetic 8 MiB decoded and 8 MiB texture pages, the former all-page model retained 16/320/1600 MiB for 1/20/100 pages. The new cold/warm, idle/scroll/zoom sequence retained 8/24/24 MiB (1/3/3 pages). The local operation timings were respectively: 1 page 311834/42/0/10083/49625 ns; 20 pages 13125/84/0/2583/17375 ns; 100 pages 7875/41/42/7500/12667 ns. These are matched optimized accounting workloads, not end-to-end Poppler, GPU or cross-platform frame-time claims; deterministic byte bounds are the regression contract.
- A fresh Catpuccin Latte main viewport framebuffer was captured and inspected after the change. The requested page is populated, aligned and unclipped. This verifies egui fallback composition only, not native webview composition.
- Architecture checklist: 38 of 38 complete, 0 remain.

3. Versioned Tinymist synchronization coordinator (2026-09-18)

- Items 191–193: a window-owned coordinator now owns Tinymist generation, current/preview URI state, open-URI membership and all private unsaved backing guards. It accepts canonical `VersionedInput` values and emits typed `Open`, `Change`, `Close` and `UpdateBacking` effects; only the app adapter performs sidecar or filesystem IO.

- Startup, edits, tab switches, parked-document admission, rename/close and shutdown now use that effect boundary. Parked documents carry their own revision instead of inheriting the active tab's version. Active/preview root selection and canonical snapshot collection also live with the policy.
- Reply admission checks generation, current URI and document revision together for formatting, hover and completion results. Tests reject each stale identity component and prove projected documents send canonical bytes while display coordinates remain editor-owned. Existing document tests prove ordinary mode builds no projection/translation state.
- The coordinator has no egui, process, thread, sidecar or filesystem-effect API. The no-consumer edit fast path remains before canonical source allocation, so disabled/unavailable Tinymist adds no typing cost. This changes ownership and stale-result safety; it adds no worker, queue, repaint loop or source copy.
- The command-line compiler limitation is intentionally unchanged: imported subfiles not represented by editor overrides are read from disk.
- Architecture checklist: 35 of 38 complete, 3 remain.

4. Stable document-record ownership (2026-09-18)

- Items 185–186: Tabs now owns one TabRecord per stable numeric ID in a keyed store plus a separate display-order list. Active and preview selection are optional IDs, so the empty workspace needs no placeholder slot or active hole. Reordering only moves an ID and cannot remap document identity.
- Every record owns its document, folding state, saved editor-widget state, workspace and autosave deadline whether active or inactive. Switching retains those records in place, rekeys revision-bound state to reject stale replies, and does not move window-owned Tinymist, compiler, indexer or native resources into tabs. Production callers use stable accessors; a boundary test rejects direct keyed-store access outside the tab module.
- Characterization and regression coverage retains preview selection, undo, cursor, dirty close, empty workspace, reorder and late-result behavior. New tests assert the keyed store/order invariant, folding and autosave retention, and that repeated switching starts no Tinymist, project-index or workspace-scan job.
- Optimized opt-in probe on Apple M2 Max, arm64 macOS 14.6.1, Rust 1.96.0: 2,000 alternating switches took 894,042 ns and requested 732,072 allocation bytes (about 366 bytes/switch, chiefly existing egui widget/workspace state). 100 idle tab-strip frames with 100 tabs took 126,145,666 ns and requested 237,217,300 bytes. This is an isolated egui layout workload, not GPU/RSS or a desktop-frame claim; normal tests keep timing out of CI. The refactor adds no worker, queue, disk read, service start or repaint loop.
- Architecture checklist: 21 of 38 complete, 17 remain.

5. Cached capability snapshot (2026-09-18)

- Item 219: one cached snapshot now reports editing, LSP, interactive preview, PDF generation, rasterization and PDF-link extraction independently from the current tool resolutions and service states. Missing pdftoppm and pdftohtml are visible separately and cannot collapse the compiler or each other into a generic failure.
- Optional executable discovery reuses toolchain.rs, runs at session startup, and repeats only for explicit Refresh tools. Typst/Tinymist preference changes invalidate only the derived snapshot; Settings-frame reads do no filesystem, environment or process work.
- Pure partial-availability and cache-invalidation tests, semantic Settings coverage and a source dependency boundary protect those rules. This adds no worker, queue or repaint loop and claims no runtime speedup. Architecture checklist: 18 of 38 complete, 20 remain.

6. Stable tab identity accessors (2026-09-18)

- Items 180 and 184: cross-tab document lookup and tab actions now use the existing stable numeric IDs. Active/preview IDs are optional for the empty workspace; positional slots are private to tabs.rs. Selection, close, rename, preview choice, save routing, dirty-close checks and navigation resolve IDs at the storage boundary instead of carrying indices across work.
- New characterization coverage proves IDs continue to resolve the same source after reorder and that repeated New-tab operations leave both the designated preview identity and source unchanged. Existing empty-workspace, dirty-close, retained undo/selection, stale-save and idle-service tests remain the contract.
- A source boundary prevents window code from reaching active/preview slots, parked storage or the old index-based document helper. This is a preparatory ownership seam for item 185, not a storage rewrite. It adds no per-frame scan, worker, source copy or repaint; no performance improvement is claimed. Architecture checklist: 19 of 38 complete, 19 remain.

7. Read-only Git views (2026-09-18)

- Item 214: the Git panel renderer now consumes immutable snapshot/status/diff input and returns a typed output batch. The app/controller applies commit-text, reveal and repository-operation actions only after section rendering. Gutter markers and hunk popups likewise return typed OpenChunk/RunHunk actions.
- The renderer owns only presentation caches: the commit buffer is copied only when the model changes or the user edits it, and diff galleys remain keyed by shared content identity, style and font-cache identity. No workers, queues, filesystem probes or repaint loops were added.
- Source-boundary and semantic tests prove Git views have no repository/worker effect handles and clicking controls emits actions without executing Git. Existing checked projected-revert coverage still restores the pre-revert text in one editor undo; repository lease/conflict tests remain passing.
- Hunk popup actions are now small buttons on the same title row as “Changes since last commit”. Shortcut text moved to hover hints to reduce width while full accessible labels remain. A deterministic geometry test checks all five controls share the title baseline and stay below 24 points high.
- All 54 focused Git tests, 9 architecture-boundary tests, full Rust tests, strict Clippy, formatting and all 13 xtask tests pass. No new runtime work is expected beyond constant-size typed outputs; no performance speedup is claimed. The semantic geometry check is sufficient for this layout change, so no screenshot is claimed. Architecture checklist: 17 of 38 complete, 21 remain.

8. Git repository service boundary (2026-09-17)

- Item 212: subprocess handling, status/diff codecs and hunk index transactions now live in `src/git/repository.rs` and its `diff/hunks` modules. Panel and editor workers use a borrowed Repository handle; UI labels and rendering stay outside the service. No extra workers, queues, source copies or idle work.
- Preserve argument arrays, literal pathspecs, bounded output, timeouts and the shared mutation lease. Disposable-repository regressions cover quoted/Unicode paths, new files, CRLF and missing final newlines, checking exact index bytes and unchanged working files. Dependency checks prevent UI ownership leaking back into the service.
- Fixed nested-workspace result provenance: mutations retain the requesting workspace separately from the canonical repository root. Direct and async panel tests ensure successful commits clear their message without a spurious refresh. Existing partial-stage, concurrency and one-undo revert tests pass.
- All 52 focused Git tests, boundary checks, full Rust tests, strict Clippy, formatting and all 13 xtask tests pass. No geometry changed; no screenshot or new performance measurement is claimed. Details: `docs/architecture-followup.md`.

- Architecture checklist: 16 of 38 complete, 22 remaining (including the measurement-gated 217). Read-only Git views remain item 214.

9. Protected background saves (2026-09-17)

- Items 189–190: active/manual/Save As/post-format and parked autosave now share one admission/completion adapter and protected worker-side disk policy. Reuse ExclusiveJob and the destination lease; no additional executor or payload queue.
- Disk-baseline verification and atomic persistence share one lease. Conflicts require fresh confirmation, uncertain writes cannot close documents, and late receipts cannot overwrite newer text or release another close continuation. Admitted writes and their outcomes survive owner closure.
- Deterministic tests cover blocked storage with runnable UI dispatch, duplicate admission, competing same-path writes, repeated conflicts, delayed Save As formatting, newer edits, replaced documents and parked/reordered tab routing.
- Matched optimized slow-storage probe: median foreground dispatch 21.792 ms synchronous versus 0.0169 ms background. Total persistence did not improve; this is a headless boundary measurement, not GUI FPS. Metadata normalization and snapshot preparation remain foreground work. Raw samples, environment and limitations: docs/background-saves.md.
- Reactivation currently rekeys document epochs, so a save receipt arriving after reactivation is conservatively rejected rather than falsely marking it clean. Stable document ownership remains items 184–186. No external-writer atomicity is claimed. Formatting, ordinary/profiling strict Clippy and full tests, and all 13 xtask tests pass; no pixel layout changed.

10. Immutable save handoff (2026-09-17)

This records the intermediate stage; items 189–190 above supersede its synchronous execution and pending-work notes.

- Item 188: src/save_transaction.rs wraps the existing projection-aware request and receipt with expected disk state, save intent, durability and continuation token. Both active and parked saves use it synchronously; no source copy, additional disk read, hash pass, queue or thread is introduced.
- Core Workflow now allocates non-reused save-continuation tokens. An earlier save cannot approve or cancel a later close request. Document receipt identity and revision validation remain solely in the existing document model.
- Tests cover canonical Unicode/miTeX bytes, pointer reuse, failed versus committed-but-uncertain writes, wrong owner/epoch, replaced continuation, duplicate completion and token exhaustion. Formatting, ordinary/profiling strict Clippy and full tests, all 13 xtask tests and boundary checks pass.
- This is correctness/testability groundwork, not a claimed latency improvement. Lease-protected disk-state rechecks and background execution remain item 189; shared scheduling/save policy remains item 190. Details and performance scope: docs/architecture-followup.md.

11. Architecture implementation batch: saves, PDF, completions (2026-09-17)

- Item 187: added adversarial tests using the real document/workflow adapter for edits during save, out-of-order same-path receipts, failed and uncertain writes, canceled close, replaced documents and one-time close continuation. These control receipt ordering; disk execution remains synchronous until 188–190.
- Item 197: moved PDF page/link types, rasterization and link extraction into src/pdf.rs, used directly by compiler, assets, thumbnails and preview views. Canonical export bytes, DPI, decoded pixels, link normalization, first-page bounds and cancellation retain focused coverage. No extra workers or copies.
- Item 203: reassessed after codec extraction. Keep Session's ordered handshake, pending requests, documents and pipes together rather than introduce competing ownership. New canceled/duplicate

reply and shutdown/replacement tests protect request identity, shutdown-before-exit and reaping-before-replacement. Existing generation/version admission and independent forced termination remain intact.

- Item 210: introduced a consumed, versioned completion transaction, with explicit canonical/display coordinates, core edit validation, miTeX preflight, resulting selection and one undo commit. Full document keys protect epoch/owner changes; regressions cover Unicode, CRLF, invalid ranges, stale transactions and undo.
- Item 211: extracted the read-only completion popup with typed selection, acceptance and dismissal actions. Semantic interaction and placement tests pass. Borrow items/source and the selected font family instead of cloning. A matched optimized payload-handoff probe reduced allocations from 1,135,250 bytes per paint to zero for 256 documentation-heavy suggestions; this excludes egui rendering and GPU costs. Normal tests enforce the allocation contract.
- Evidence, ownership rationale and raw probe metadata: docs/architecture-followup.md. No fresh visual claim: layout is unchanged. Formatting, strict ordinary/profiling Clippy, both full test suites, all 13 xtask tests and both real-Tinymist tests pass.

12. Architecture todo review (2026-09-17)

- Item 207: ProjectIndex now retains unreadable paths/error kinds and an explicit traversal-cutoff flag. Leftover duplicates/cycles do not produce false cutoff warnings at exactly 256 visited files. Existing dynamic-dependency records feed the same concise partial-index summary, with no second dependency model. Explorer displays the wrapping summary once above its sections, including while searching; a recovered index removes it. Missing/unsaved paths now canonicalize their existing ancestors so symlinked workspace paths do not silently exclude them or lose their in-memory overrides.
- Item 207 verification: new deterministic tests cover exact/over-limit traversal, duplicate/cyclic references, missing entry/imported files, invalid UTF-8, override recovery, dynamic dependencies, package exclusions and symlink aliases. A semantic UI test covers warning visibility during an empty search, narrow wrapping and removal after recovery. Required formatting, strict Clippy, full normal tests (800 application passes, 8 opt-in tests ignored) and 13 xtask tests pass. No screenshot is claimed; behavior and wrapping have deterministic tests.
- Item 207 resource impact: no extra indexing jobs, repaints or package downloads. Failure records are bounded by the existing 256-file traversal limit; summary aggregation happens once in the worker and rendering borrows the cached text. Missing-path normalization performs ancestor checks only after canonicalization fails. No material normal-frame performance change or measured speedup claimed.
- Item 213 implemented after review: panel writes and Stage/Unstage hunks now use one repository-root transaction entry point. Root discovery precedes the lease; status, HEAD, index and patch preconditions are read under it. Nested workspace aliases resolve to the same lease. Navigation/Revert actions cannot enter the index-write path. Revert remains an editor edit and the existing ExclusiveJob protected-completion mechanism is unchanged. External Git writers still rely on Git's own locking/patch validation, not this process-local lease.
- Item 213 verification: all 48 focused Git tests pass, including new simultaneous two-window hunk plus panel staging, locked baseline revalidation, real merge conflict rejection and index preservation. Tests assert that mutating paths hold the resource lease; existing quoted-path/new-file/CRLF/no-final-newline, partial-stage and one-step projected-undo regressions remain passing. Required formatting, strict Clippy, full normal suite and all 13 xtask tests pass.
- Item 213 resource impact: the added lease is worker-side and action-triggered; no idle repaints, per-frame work or extra workers. Hunk root/status discovery is not duplicated, and panel lock discovery

no longer needs a full status/log snapshot. No wall-time speedup claim or unrelated GUI benchmark; UI pixels and native composition are unchanged. Item 207 is the next small correctness task.

- A01 (184–186): justified by active-hole/parallel-vector invariants, not by an absence of IDs. Reuse current IDs and tests; no rope or per-tab service rewrite.
- A02 (187–190): worthwhile because saves still perform disk work synchronously. Existing core requests/receipts and writer leases are assets to reuse. When moving expected-state checks under the lease, avoid recursively acquiring the same non-reentrant writer lock. Preserve durable-close rules before optimizing.
- A03/A04 (191–196): useful only if extraction removes duplicate orchestration. Existing version admission, recovery and content models remain authoritative; do not build a generic event bus or mirror state in a new coordinator.
- A05 (197–200): strong resource-efficiency case: the raster viewer retains all pages while culling only drawing. Separate metadata before introducing bounded residency, and measure multi-window resource limits. Export remains untouched.
- A06 (202–203): codecs are a clear pure boundary. Reassess further session splits after that move; file size alone is not a reason for another abstraction.
- A07 (204–206): real concern: LatestJob replaces receivers but spawns work that can continue. Start with indexing and explicit bounds, not a general scheduler.
- A08 (207–209): incomplete scans silently skip unreadable files/cap overflow, so honest completeness reporting is a small immediate win. Shared snapshots and filesystem notifications need reliable invalidation and a tested fallback; they must not share window-local filters or unsaved buffers accidentally.
- A09 (210–211): one completion transaction/view seam is reasonable. Reuse core coordinates/projection preflight; do not rewrite TextEdit, IME or every popup.
- A10 (212–214): the repository lease gap is a correctness issue independent of code organization. Implement 213 first; later service extraction must preserve checked patch semantics and protected completions rather than replace Git.
- A11 (215–217): owner cleanup is justified by viewport-keyed retained font slots. Property diffing is only a hypothesis and is now measurement-gated. Hidden and closed native views must remain distinct.
- A12 (219): capability reporting is useful, but derive it from current resolution and service state instead of adding a second source of truth. Completed items 218/220/221 remain valid and are not reopened by this review.

13. Architecture audit implementation: first small changes (2026-09-17)

- Item 221: Corrected the enabled miTeX API comments and added a dated ADR 0002 amendment plus an ownership/module map. Original rationale remains historical; current notes describe independent document/Settings viewports, per-window TinyMist, conditional CLI work and the imported-subfile limitation.
- Item 220: Build-time metadata now comes from `toolchain/manifest.tsv`, replacing duplicate runtime version literals. The shared validator rejects missing tools, duplicate targets, malformed rows and cross-target version disagreement. Three integration tests cover generation and rejection. No runtime manifest IO or changes to executable selection, bundled binaries or compatibility behavior.

- Item 218: Moved launch types, argument/environment parsing and persistence policy into `src/launch.rs`; the capture renderer no longer owns application startup. Explicit profiling mode preserves session-owned storage. Normal profiles can persist only to that isolated store; capture profiles still do not persist QA fixtures. Existing capture/CLI tests and new mode/storage tests pass. Capture naming, scene routing and pixel output contracts are unchanged.
- Item 201: Extracted JSON-RPC framing into `src/tinymist/transport.rs` without moving workers or changing sidecar commands/events. Also fixed the header limit: previously checked only after an unbounded line read, it now limits the read itself. Deterministic regressions assert at most 8,193 bytes consumed for an overlong header line and 32,769 for oversized aggregate headers, including the single overflow-detection byte. Invalid lengths fail before payload allocation. Existing framing, process-shutdown and stale-session tests remain intact.
- Verification: required formatting, strict Clippy, normal tests and 13 xtask tests pass; profiling-feature checks also run for the launch-policy change. Both real Tinymist preview-cycle and unsaved-formatting probes pass with `TIPTOPTYP_TEST_TINYMIST` explicitly selecting the bundled macOS arm64 binary. The initial preview probe lacked that required environment selection and failed before starting its server; rerunning with the explicit path succeeded.
- Performance: no new idle repaints, per-frame IO, worker queues or payload copies. Version generation runs only at build time; launch parsing remains startup-only. Framing resource bounds are tested deterministically, not presented as a measured interaction-speed improvement. No pixel/native geometry changes or visual verification claimed. Remaining audit tasks stay open; item 202 is next.

14. resolved in the 2026-09-13 easy backlog pass

- Status/Git follow-up (2026-09-16): color-code compact +/- line totals with the same added/modified/deleted theme colors as the gutter, keeping one shared status-message location and history.
- Explorer active-file typography follow-up (2026-09-16): active filenames now inherit the same UI text size as neighboring rows, retaining only heavier weight. Regression tests cover multiple UI sizes and row heights; fresh active/neighbor framebuffer inspected. Rust checks pass. Full gallery refresh timed out at 113 seconds and restored the previous 68 images; the restored gallery validates, but is not new visual evidence.
- Items 121, 122: Explorer asset hover candidates now end at the visible panel clip, and the active file's stronger face keeps the shared content font size.
- Items 123, 124: Preview status text uses "Preview ready" consistently and suppresses zero-millisecond timing so startup fixtures cannot claim a completed PDF build.
- Items 125, 126: Find query edits select the first result immediately, and every result receives a syntax-preserving background highlight with a stronger color for the selected match.
- Item 127: Main-window popups no longer close just because a settings, package, Git, or chunk child window is visible; only overlays owned by the root editor can block them.
- Items 128, 129: File diffs toggle closed from the same row action, and hunk diffs now render in the owning editor's popup with Escape or dismissal closing the chunk; the separate Git chunk child viewport is gone.
- Item 130: Normal document windows render the Git panel as an Explorer subpanel; the separate Git child remains only for the deterministic Git window scene. View > Git also opens Explorer when it was closed, and a one-shot reveal overrides a stale collapsed section state.
- Item 138: Tooltip code blocks cache their highlighted LayoutJobs per document viewport and theme, avoiding repeated syntax work while scrolling a hover popup.

- Item 139: Git hunk markers now reserve an eight-point lane, with painted and clickable geometry constrained to that lane.
- Verification: Added explorer geometry, active-font-size, startup timing, layout-highlight, popup gating, diff-toggle, popup-routing, tooltip-cache, and compact-gutter regressions; focused tests pass.

15. resolved in the 2026-09-15 pass

- Item 141: Tinymist timeouts and fatal server failures now restart the sidecar automatically, retaining the interactive preview during retries; the raster fallback is selected only after five consecutive failures. Recovery resets the failure count once the server is ready.
- Verification: Tinymist retry-budget regression, focused Tinymist tests, full Rust tests, clippy, formatting, and xtask tests pass.

16. resolved in the 2026-09-07 pass

Items 1, 2, and 26 remain deferred. This history records completed work and partial progress; the running checklist above is authoritative for completion status. Focused tests cover the deterministic behavior described below:

- Items 3, 4: Typst overrides now live under Appearance, show the effective color, and use Reset to return to inherited theme values.
- Item 6: Removed redundant hover cards from override labels, color swatches, live samples, and Reset buttons; decoration controls retain only concise state-and-action hints.
- Items 7, 8, 9: Built-in theme selections show palette swatches; UI font choice supports the default, editor-monospace, or a validated custom TTF/OTF/TTC file, with interface scaling via Cmd-plus/minus.
- Items 10, 14, 15: Markdown tooltip blocks use syntax highlighting; typ, typst, and typc fences use the Typst highlighter.
- Items 12, 13, 58: Tooltip cards are wider, scrollable, clipped to the viewport, retain a root-local pointer bridge to the card, and support click or Cmd+Shift+Space focus, selectable copying, Escape dismissal, and a focused outline.
- Items 5: Preview geometry is clamped to the actual panel clip and zoomed previews preserve their viewport origin.
- Items 19: External edits reload clean editable documents and produce an explicit conflict notice for dirty documents.
- Items 24, 25, 29, 30, 31, 51: View commands, shortcuts, title-bar menu visibility, find/replace toggling, replace actions, case sensitivity, regex mode, and the regex help menu are wired through shared commands.
- Items 27: The find/replace bar is a foreground overlay and no longer reserves editor layout space.
- Items 32, 34: Explorer context menus can open files in a new window and toggle the designated preview file on and off.
- Items 35, 53: Cmd-slash toggles comments for the current or selected lines; editor context menus can reveal the source in Preview.
- Items 37, 39, 46: Problems is the rightmost view control; file labels use theme colors by document category; CJK fallback fonts are registered when available.
- Items 45: Tags and references are indexed and navigable from a new Explorer section.
- Items 48: macOS child windows use native traffic lights; non-macOS windows retain an in-content close action.
- Items 61: Renaming a designated preview entry recreates its Tinymist URI safely.
- Items 60: Saving an existing Typst file requests formatting when Tinymist is ready and persists the formatted source; auto-save does not format. The initial Save As flow still needs formatting integration.

- Items 61: A designated preview remains alive while editing child Typst/text/PDF/image files, and its state is restored when returning to source.
- Items 47, 63, 64: Status history has timestamps, dirty-title space is reserved, and the main title bar uses the settings `ttt` logo.
- Items 62: Small-text styling was removed from the application UI; supporting text uses the shared readable type scale.

Open work is tracked only by the unchecked entries in the running list above, so this audit history cannot become a second, stale task list.

Item 68: Native screenshot capture was initially blocked by the sandbox's `LaunchServices/Services` connection. Gallery decoding and manifest tests passed; fresh capture subsequently succeeded during the audit below.

17. correctness audit (2026-09-07)

- Items 5, 9: Corrected UI scaling to preserve native display density, with a 1x/2x display regression test.
- Items 31: Replaced the recursive regex parser with the maintained regex-automata engine already used by Syntect. Added Unicode, invalid-pattern, zero-width navigation, and pathological-pattern regressions. Replacement text remains literal.
- Items 19, 27, 28: Fixed external-change polling when Explorer is hidden, kept find overlays anchored to the visible editor, and preserved find-field focus while navigating matches.
- Items 60: Prevented stale formatting replies from cancelling a newer save request; cleared pending save-format intent on edits and document switches.
- Items 61: Preserved current preview-entry edits queued before Tinymist initializes and removed a duplicate restart when renaming the current preview entry.
- Items 10, 14, 15: Shared bundled syntax databases across tooltip highlighters and corrected Typst tooltip colors in light mode.
- Item 8: Validated custom TTF/OTF/TTC bytes before loading, retained a fallback chain, and kept editor syntax fonts independent from the selected UI font.
- Items 45: Used the Typst syntax tree for labels/references, excluding strings, comments, and raw examples.
- Items 61, 68: Isolated screenshot fixtures from persisted designated-preview settings without changing those settings.
- Items 5, 9, 27, 68: Native capture succeeded outside the sandbox. Inspected the fresh find/replace PNG under `.tiptotyp/screenshots/agent-review`; retained native bounds and Retina scale evidence in `audit-geometry.txt` alongside it.
- Items 68: Regenerated and validated all 68 maintained gallery PNGs; inspected representative light/dark editor, find overlay, and settings captures. Formatting, strict clippy, 308 application/integration tests, and 7 packaging tests pass (2 environment-dependent tests remain ignored).

Open work is represented by the unchecked entries in the running list above.

18. correctness audit (2026-09-07 tooltip follow-up)

- Items 12, 13: Native tooltip geometry now converts the monitor-space child position back to root-local coordinates for pointer bridging. Click or `Cmd+Shift+Space` requests popup focus, Escape or defocus dismisses it, and the focused card gets a subtle outline.
- Items 10, 14, 15: Selectable tooltip labels make rendered markdown copyable. `typc` fences parse as synthetic `#{...}` code-mode snippets, strip the synthetic delimiters, and are covered for keyword, number, and function roles.

- Items 12, 13, 10, 14, 15, 68: Fresh diagnostic and function tooltip screenshots were captured and inspected in Catppuccin Latte and Mocha. The full serialized suite passes with strict Clippy and formatting checks.

19. correctness audit (2026-09-07 final follow-up)

- Items 12, 13: Retained diagnostic tooltip payloads while the pointer crosses the root-to-native-card bridge, so the popup no longer disappears during transit.
- Item 19: External-file polling now distinguishes a missing file from an unreadable file and reports deletion explicitly for both clean and dirty buffers.
- Item 61: Renaming a non-preview child document no longer tears down a live designated Tinymist preview; renaming the designated entry itself still recreates its URI intentionally.
- Items 29, 46: Find toggling is consistent across the shortcut, toolbar, native Edit menu, and title-bar Edit popup; project indexing and workspace colors now accept the same case-insensitive Typst extension policy as document detection.
- Verification: The corrected suite passes with 308 application/integration tests, 2 ignored environment-dependent tests, 7 packaging tests, strict Clippy, formatting, diff checks, 68 PNG decodes, and a fresh inspected find/replace capture.

20. correctness audit (2026-09-07 tooltip handoff correction)

- Items 12, 13: Replaced the center-point triangle with an edge-to-edge pointer corridor, so paths toward the tooltip's top or bottom edge remain connected during the hover handoff.
- Verification: The tooltip bridge regression now covers a top-edge transit path; focused tooltip and coordinate-conversion tests also pass, and a fresh diagnostic tooltip PNG was captured and inspected.

21. correctness audit (2026-09-07 competing tooltip handoff correction)

- Items 12, 13: Prevented another native hover source from replacing the active tooltip while the pointer is inside the original tooltip's bridge or focused popup. This covers diagnostic, semantic editor, and shared native-control hover payloads.
- Verification: Added a competing-hover regression covering bridge retention, focused retention, and dismissed cleanup; the full suite, strict Clippy, formatting, diff checks, and a fresh inspected diagnostic tooltip capture pass.

22. correctness audit (2026-09-08 tooltip opacity correction)

- Items 12, 13: Limited handoff blocking to competing source rectangles, allowing the active source to continue its fade-in animation to full opacity while still protecting its payload during transit.
- Verification: Added same-source versus competing-source assertions; focused tooltip, bridge, Clippy, formatting, diff checks, and a fresh inspected diagnostic tooltip capture pass. One serialized full-suite run hit a transient fake-Tinymist handshake timeout; the failing test passed when rerun alone.

23. correctness audit (2026-09-08 tooltip frame correction)

box

- Items 12, 13, 58: Sized native tooltip viewports from the tooltip frame's complete inner, outer, and stroke margins, preventing the lower rounded edge from being clipped.
- Verification: Tooltip and popup-frame tests, strict Clippy, formatting, diff checks, and a fresh inspected diagnostic tooltip capture pass.

24. correctness audit (2026-09-08 native popup edge correction)

- Items 12, 13, 58: Short native tooltips now shrink their vertical scroll area to the rendered content, so the lower rounded frame remains visible instead of reaching the child viewport edge.
- Items 12, 13: Pointer and focus hit-testing now uses the painted frame rectangle, excluding transparent child-viewport padding from hover handoff and clicks.
- Item 68: Native menu content budgets now derive from the rendered frame margins rather than hard-coded chrome values; the screenshot workflow now explicitly documents that viewport PNGs are not composed desktop screenshots.
- Verification: Added a popup content-budget regression, captured and inspected the exact single-line tooltip case and the File menu in a fresh Catpuccin Latte release run, and retained the native preview geometry trace.

25. correctness audit (2026-09-08 tooltip handoff regression correction)

- Items 12, 13: Restored the full transparent native child viewport as the pointer handoff envelope while keeping painted-card hit-testing precise, preventing a competing tooltip from taking over before the pointer reaches the active card.
- Verification: Added a regression for competing targets across the child viewport, passed the focused tooltip tests, and captured and inspected a fresh traced Catpuccin Latte release tooltip framebuffer.

26. correctness audit (2026-09-08 robust tooltip handoff correction)

- Items 12, 13: Closed the semantic-editor hover path that bypassed the shared native-tooltip handoff gate, and added identity-preserving geometry plus a 300 ms grace window so a transient pointer gap cannot dismiss the active tooltip before the pointer reaches it.
- Verification: Added a transient-gap regression, passed 7 focused tooltip tests, passed the full 286-test application suite plus integration and 7 packaging tests, and captured and inspected fresh release tooltip framebuffer 1788801929503-0001-diagnostic-diagnostic-tooltip.png.

27. correctness audit (2026-09-08 child-owned tooltip handoff and syntax correction)

- Items 12, 13: Made the native tooltip child viewport the sole owner of its pointer-presence state. Root-window pointer gaps can extend the route deadline but can no longer clear a child-owned hover, focus-transfer frames remain renderable, and identity checks prevent a closing tooltip from mutating its replacement.
- Items 10, 14, 15: Converted language-tagged Tinymist marked strings into real fenced Markdown, routed typ and typst through Typst source mode and typc through synthetic `#{...}` code mode, and syntax-highlighted inline code without leaking its style past a closing backtick.
- Items 10, 12, 13, 14, 15, 68: Added regressions for child-owned pointer state, stale tooltip identities, focus transfer, source-versus-code highlighting, and inline marker closure. Formatting, strict Clippy, 291 application tests (289 passed and 2 environment-dependent ignored), integration tests, 7 packaging tests, and all 68 PNG decodes pass. Fresh Catpuccin Latte and Mocha diagnostic and function tooltip framebuffers were captured and inspected.

28. correctness audit (2026-09-08 pre-commit)

- Items 24, 29, 34, 51: Ordered modified shortcuts before their generic variants, preventing Open, Find, and interface scaling from consuming Open in New Window, Change Workspace Root, Find and Replace, or raster-preview zoom. Raster zoom now uses Cmd/Ctrl+Option/Alt with Plus, Minus, or 0 so item 9 keeps the unmodified Cmd/Ctrl+Plus and Minus contract.

- Item 28: Enter and Shift+Enter now move forward and backward respectively while returning focus to the find field; the broader cross-field shortcut audit remains open.
- Item 19: External-file observations are cleared on document switches and successful saves, preventing one file's missing/changed state from suppressing another file's notification.
- Item 60: Manual saves confirmed through the external-change overwrite dialog now request formatting too, while save-before-close/open flows remain immediate. Tinymist restarts invalidate outstanding save-after-format intent.
- Items 12, 13: Stale tooltip interaction state is identity-checked and cannot block all future tooltip sources when native geometry is temporarily absent.
- Items 39, 46, 63: Added the omitted ICO image category, made UTC status timestamps explicit with Z, and avoided reapplying UI font/style configuration every frame.
- Item 68: Added shortcut, timestamp, file-category, and tooltip-state regressions. Formatting, strict Clippy, 295 application tests (293 passed and 2 environment-dependent ignored), 26 integration tests, and 7 packaging tests pass. All 68 maintained PNGs validate; fresh Catppuccin Latte and Mocha find, diagnostic-tooltip, and status-log captures were inspected, with final native tooltip/preview geometry retained in audit-geometry.txt.

29. resolved in the 2026-09-08 font and embedded-syntax pass

- Item 11: UI fonts now expose their real OpenType weight range or installed static faces, while editor syntax roles use numeric weights from 100 through 900. Typst override settings migrate the old bold flag to weight 700 without losing saved themes.
- Item 16: String and raw-string arguments to MiTeX math/text entry points and Cmarker render entry points now use embedded TeX or Markdown highlighting while preserving exact editor byte positions.
- Item 17: Literal arguments to Typst's RGB, luma, CMYK, linear RGB, HSL, HSV, Oklab, and Oklch constructors now show a readable preview fill. Detection is limited to Typst's actual namespaces and valid literal component types so unrelated custom calls are unchanged.
- Audit for items 11, 16, and 17: Corrected a clipped Settings weight row, removed axis-relative slider rounding that changed 400 to 401, kept deterministic screenshot runs from persisting QA-only settings, made the offline Cmarker fixture valid Typst, and tightened invalid color-call detection. Formatting and strict Clippy pass; 302 application tests pass with 2 environment-dependent tests ignored, alongside 26 integration/gallery tests and 7 packaging tests. All 68 maintained PNGs decode and validate; fresh Catppuccin Latte and Mocha main, Settings, and Typst-overrides viewport PNGs were inspected, with traced native preview bounds remaining clipped to the panel.

www.google.com

30. correctness audit (2026-09-08 fonts, popups, and greenfield cleanup)

- Items 20, 72, 74, 76: Code ligatures are enabled. Settings has separate system and workspace font dropdowns and independently persisted UI and code weights. Continuous weight changes are staged until pointer release, and both slider rows share one vertical-centering contract.
- Items 55, 56: Right-clicking a text font argument opens a scrollable document-font selector. Workspace fonts are discovered asynchronously and supplied to Typst and Tinymist. File fingerprints catch additions and in-place replacements while preserving the prior catalog during a rescan.
- Items 63, 73, 79: Preview status and visible notices share a timestamped newest-first log capped at 100 entries. The popup grows only for visible rows, hugs the status bar, and uses a fixed right-aligned timestamp column.

- Items 69, 78: Popup state now has one owner. Settings and app menus close competing transients, blur requires a grace period, each menu opening has independent scroll memory, and popups paint immediately at full opacity.
- Items 75, 77: Settings palette swatches are gone. Typst override headers, syntax labels, samples, and Reset buttons share exact row centers. Deterministic Settings and override scenes reset their scroll origins.
- Item 80: Removed the obsolete app bundle, registration, build products, scratch previews, deprecated names and environment variables, old settings migrations, deprecated LSP root path, Finder process-serial handling, and disconnected editor receiver plumbing. The current settings schema is a clean break.
- Item 26 remains deferred.
- Items 68, 72 through 80: Regenerated and decoded all 68 gallery PNGs and inspected fresh Catppuccin Latte and Mocha Settings, theme-picker, menu, tooltip, status, font-selector, and Typst-override captures. The native inspector did not expose the current bundle window, so viewport captures are not claimed as proof of composed desktop geometry.
- Verification for items 68, 72 through 80: Formatting, strict Clippy, 318 application tests (316 passed and 2 environment-dependent ignored), 26 integration tests, and 7 packaging tests pass. Fake-Tinymist test deadlines now tolerate full-suite scheduler load without changing runtime timeouts. Diff checks pass and no obsolete pre-rename namespace, deprecated root-path field, or removed alias remains in tracked implementation.

31. correctness audit (2026-09-08 pinned preview and risk-based UI evidence)

- Items 61, 70: The designated Typst entry is passed to both Tinymist compilation and startDefaultPreview. Switching among project Typst, text, PDF, and image files preserves the same native WebView and only updates the active LSP document. A replacement preview server now forces one navigation even when the operating system reuses its predecessor's localhost URL.
- Items 61, 70: Interactive preview suppresses duplicate raster compilation during steady-state use. Raster pages remain an explicit fallback for unavailable native preview, export, and targeted framebuffer capture.
- Items 68, 81: AGENTS.md and its regression test now require screenshots only for materially visual risks. The maintained gallery remains a serialized one-app-session workflow, but no fresh capture was performed in this audit at the user's request.
- Verification for items 61, 68, 70, and 81: Formatting, strict Clippy, the full Rust suite, all decodable gallery/manifest checks, and 7 packaging tests pass. Items 61, 70, and 81 remain unchecked until their explicitly outstanding manual exercises are completed.

32. correctness audit (2026-09-08 vector icon redraw)

- Item 82: Replaced the overlapping refresh chevron with a continuous arc-and-arrow path whose outer wing remains outside the circular body. Replaced the straight-sided preview eye with symmetric cubic Bezier curves and a centered round pupil.
- Verification for item 82: Deterministic geometry tests cover arrow/body separation and curved-eye symmetry. Formatting, strict Clippy, and the full Rust suite pass. No app launch or screenshot was used; item 82 remains unchecked for user visual verification.

33. correctness audit (2026-09-08 external link interactions)

- Item 83: Tooltip Markdown links now omit their destination syntax, retain selectable text, and dispatch normalized HTTP(S) targets through the shared system-browser route. The editor recognizes literal external targets in real #link(...) calls: normal clicks remain editing actions, Cmd+click opens

the link, and right-click exposes an Open Link in Browser action. Raster and native preview links use the same route.

- Item 83: Native preview navigation callbacks now read the current preview URL, project root, and designated-source directory from shared session state. Reusing a pinned WebView after Tinymist restarts therefore cannot mistake the replacement localhost origin for an external page or resolve project links relative to a stale editor file.
- Verification for item 83: Parser, Unicode cursor mapping, unsafe-scheme rejection, command-click priority, semantic tooltip activation, semantic context-menu activation, popup sizing, project-file routing, external preview dispatch, and replacement-origin routing have deterministic regressions. No app launch or screenshot was used; item 83 remains unchecked for user interaction verification.

34. correctness audit (2026-09-08 inert chrome and explorer copy actions)

- Item 84: File-tree names, every ttt logo instance, and the main title-bar filename now explicitly opt out of egui label selection while document-like labels remain selectable. File context menus expose separate commands for the file name, absolute file path, and path relative to the workspace root; directory menus retain their existing Copy Path command.
- Item 84: The native View menu now advertises the same Cmd+1 through Cmd+5 mapping already handled by the editor and shown in the title-bar View popup: Explorer, Code, Split, Preview, and Problems.
- Verification for item 84: Selection behavior, clipboard text derivation, context-menu routing and sizing, and native shortcut metadata have deterministic regressions. No app launch or screenshot was used; item 84 remains unchecked for user interaction verification.

35. correctness audit (2026-09-08 stable explorer activation)

- Item 49: Opening another document inside the current workspace now preserves the cached filesystem snapshot instead of clearing it or scheduling an unnecessary scan. The stable snapshot generation keeps the tree identity, expansion state, and scroll memory intact; a real workspace-root change still installs a new snapshot.
- Item 50: The active Explorer entry retains the shared active-row background and now uses a separately registered heavier face of the selected UI font. This makes the current file genuinely bold instead of relying on egui's stronger-color-only strong flag.
- Verification for items 49 and 50: Workspace invalidation and the distinct strong UI font role are covered programmatically. No app launch or screenshot was used; both items remain unchecked for user interaction verification.

36. correctness audit (2026-09-08 compilation controls)

- Item 85: The toolbar now has an explicit Pause/Resume control. Pausing cancels queued automatic work, stops the persistent CLI watcher, suppresses editor-driven Tinymist updates, and leaves one-shot PDF exports and deterministic captures available. Resuming sends the newest editor buffer and restarts whichever preview pipeline is required.
- Item 85: Compile and Cmd+R now materialize the canonical PDF beside the effective saved Typst entry, including a designated preview entry while another file is being edited. Unsaved Typst documents ask for their first output location; Export PDF remains the choose-another-destination command. Compile and Export retain distinct chooser, queued, completion, and cancellation messages, and a second output request can no longer overwrite an already queued destination.
- Item 85: Completing a one-shot Compile or Export while automatic compilation is paused now reaps the temporary watcher again instead of leaving it active after producing the PDF.

- Verification for item 85: Deterministic tests cover pause gating, explicit-build exceptions, watcher shutdown, toggle copy, current/designated output paths, output intent, and unsaved/non-Typst rejection. Formatting, strict Clippy, all 372 non-ignored repository tests, all 7 packaging tests, and the ignored real-typst watcher compile/error/recovery integration test pass. No app launch or screenshot was used; item 85 remains unchecked for user interaction verification.

37. completed non-deferred backlog and startup correction (2026-09-09)

- Items 18, 21, 23: Added an asynchronous searchable package inspector for available and every locally installed release/path, recursive search across all Explorer sections and every individual Settings control, and bounded image/PDF previews for editor literals and Explorer files.
- Items 22, 28, 36, 38: Added one configurable shortcut model for all application actions and every owned text-edit viewport. Effective bindings now drive menus and tooltips; semantic clipboard events respect rebinding, delayed native paste is admitted once within a bounded window, invalid plain-letter bindings cannot steal typing, and reassignment removes hidden collisions.
- Items 33, 41 through 44, 49, 50, 52, 54, 65: Made application chrome inert while preserving selection in document content, gave the active file a readable current-line background and true heavier font, unified native/title-bar menus, added draggable Explorer sections and a one-frame content hide, preserved tree/scroll identity, added clickable line-numbered sticky context, left-aligned fallback menus, and made real Problems-row double-clicks navigate.
- Items 40 and 57: Integrated versioned Tinymist completions with snippets, additional edits, stale-response rejection, keyboard navigation, and atomic undo; added conservative syntax-tree table detection plus a modal row/column/cell editor whose changes apply as one Unicode-safe replacement.
- Item 59: Enabled crate-wide unsafe-code denial and retained only four narrowly scoped native adapters, each with an adjacent safety contract and an architecture inventory.
- Items 60, 61, 70, 71: Manual Save and Save As format the exact destination Typst document while auto-save remains unformatted; a designated preview entry survives unrelated document switches and refreshes without recreating its WebView; standard macOS window shortcuts work, and native Quit routes through the process-wide dirty-document guard.
- Items 66, 67: Added canonical ttt vector/raster/ICO artwork, embedded the runtime icon before native window creation, and included the icon contract in packaged builds so direct and packaged launches retain it through shutdown.
- Items 81 through 84: Captured the complete 68-image maintained gallery in one app session; verified the redrawn refresh/preview icons, external Markdown/Typst/preview link routing, inert tree/logo/title labels, three file-copy actions, and Cmd+1 through Cmd+5 native View-menu metadata.
- Item 85 correction: Pause/Resume controls automatic preview refresh only. Paused documents continue sending Tinymist LSP changes so completion, diagnostics, and formatting remain live. Compile/Export waits for a strictly newer artifact after pending edits and writes the effective saved Typst entry beside its source, using a picker only where required.
- Item 86: Removed the startup deadlock by deriving viewport-scoped egui IDs before entering context data/memory locks. Compiler, rasterizer, and Tinymist pipe readers now also have bounded shutdown after their direct child exits, so a wrapper descendant retaining stdout/stderr cannot wedge stop, restart, or application exit.
- Verification: Formatting, strict Clippy, and diff checks pass. The repository suite passes 540 tests with 2 explicitly environment-dependent tests ignored; all 8 packaging tests pass. All 68 maintained PNGs were freshly generated and decoded, representative light/dark main, Settings, menu, tooltip, and icon captures were inspected, and a final traced native launch painted, captured, reported in-bounds preview geometry, and exited normally.

At that point, only deferred items 1, 2, and 26 remained open.

38. correctness audit (2026-09-12 easy backlog)

- Items 88, 89, 90, 91, 92, 96, 101, 102, 103, 104, 105, and 107: Added the Finder/file-manager action to the Explorer menu, reused a live TinyMist session for in-place external reloads, recorded successful browser launches as green status notices, normalized malformed variable-font axes, placed asset previews to the right of Explorer rows, disabled non-Typst view-mode controls, added Packages and Rename to title-bar menus, exposed package website links, surfaced installed packages before the registry fetch, inserted line comments at column zero, and painted Find/Replace after the sticky context overlay.
- Verification: Formatting, strict Clippy, 517 application tests (2 ignored), 26 integration tests, and 8 packaging tests pass. New regressions cover variable-axis normalization, popup actions, package-local loading, external link notices, comment placement, view-mode availability, and sticky/find geometry.

39. correctness audit (2026-09-12 remaining active backlog)

- Items 87 and 94: Explorer drops copy regular files into the hovered folder without overwriting existing entries; dropping onto the editor opens files. Explorer deletion requires confirmation and handles the current document and designated preview entry. Filesystem regressions cover collisions, source preservation, directory boundaries, and symlinks; UI tests cover drop routing.
- Item 93: Cmd-Shift-Space toggles the tooltip under the mouse; Cmd-Shift-K opens it at the caret. Both shortcuts are configurable. Focused cards support scrolling and Escape dismissal, with dismissal retained until rearmed or the pointer leaves the source.
- Item 95: View > Git opens workspace-scoped status, diffs, staging, unstaging, commits, recent history, fetch, fast-forward pull, and push. Operations use files on disk and configured remotes and credentials. Tests exercise temporary repositories and a local remote, including rejecting divergent pulls without discarding local changes.
- Items 97, 98, and 106: Completions fuzzy-filter as the prefix changes, with Unicode-safe rebasing and filtering before the result limit. Font completion remains available after a space or opening quote. Reference suggestions show insertion code before descriptive metadata and filter on that code. Member completions filter the member name independently of its receiver.
- Item 99: Completion selections and Settings font-picker hover cards render a sample using the actual font file. Loading happens in the background and replaces a bounded picker slot. Font pickers also support fuzzy search.
- Item 100: Each installed package version has an Uninstall action, a directory confirmation, release-path validation, and an immediate installed-list refresh. Tests verify that sibling versions and linked directories remain untouched.
- Item 108: Ready image/PDF hover cards contain only the image. Loading and failure states retain their explanatory messages.
- Item 109: Mandatory command descriptors generate both native and title-bar File/Edit/View menus. Semantic tests exercise every shared command. Menu geometry tests cover all rows, separators, and changing from a shorter menu to a taller one without retaining the previous scroll clipping.
- Verification: Formatting, strict Clippy, and diff checks pass. The repository suite passes 583 tests, with 2 environment-dependent tests ignored; all 8 packaging tests pass. The complete 68-image gallery was regenerated in one app session and validated. Fresh targeted font, asset, and menu captures under `.tiptotyp/screenshots/agent-review` were inspected in Latte and Mocha. The final editor-context capture after font-preview scenes reproduced missing glyphs in one disabled label; the stable gallery's corresponding label renders correctly. This remains tracked by deferred item 26, not treated as resolved.

Only deferred items 1, 2, and 26 remain unchecked.

40. correctness audit (2026-09-12 font-preview flashing)

- Items 99 and 110: The previous preview implementation called `egui's set_fonts` when the highlighted family changed, invalidating the shared font atlas and cached text layouts across the application. Samples now use an isolated, worker-owned font atlas uploaded as a separate texture, with four recently used samples cached per picker. Browsing fonts never changes the application's font definitions. Font files are released after rasterizing.
- Items 26 and 110: Font-preview flashing was a regression in the new preview implementation. The earlier observation should not have been attributed to deferred item 26 without tracing it. This specific trigger is fixed; item 26 remains open for any unrelated intermittent text corruption.
- Verification: A regression repeatedly switches samples while preserving custom UI/code text-layout identities and asserting that no shared-font texture updates occur. Separate geometry tests cover different actual fonts, 1x/2x resolution, and normalized texture coordinates. Formatting, strict Clippy, all 584 repository tests (2 environment-dependent tests ignored), and all 8 packaging tests pass. Five fresh native viewport captures were inspected: font completion and Settings samples in Latte/Mocha, followed by the editor context menu. The previously incomplete menu label renders fully.

41. correctness audit (2026-09-13 Git window)

- Item 111: Git now has section headings, file counts, consistent right-aligned action columns, truncated paths with full-path hints, and a dedicated, selectable diff viewer with colored additions and deletions. Comparisons explain staged versus unstaged changes, including loading, empty, and error states. Finished comparisons scroll into view even in a short window.
- Git jobs are collected in the Git child viewport as well as the editor. Previously the child depended on the editor repainting to collect results, and an empty diff appeared as a generic operation-success message.
- Unstage all removes index changes while preserving working files, including files edited again after staging in a repository without its first commit. Stage all excludes app-owned `.tiptotyp` artifacts and handles literal paths through a NUL-delimited `pathspec` file. Already staged artifacts stay visible for unstaging; no existing repository index was changed during this work.
- Verification: All 11 focused Git tests pass, including real temporary Git repositories, child-only async completion, narrow/wide action alignment, short-window diff reveal, empty/error states, and file preservation. Formatting, strict Clippy, all 591 repository tests (2 environment-dependent tests ignored), and all 8 packaging tests pass. One sidecar-shutdown test failed on the initial full run, passed in isolation, and passed in the final full run. Fresh Git viewport captures were inspected in Latte and Mocha; the 68-image gallery was regenerated in one app session and validated.

42. correctness audit (2026-09-13 Git editor integration)

- Item 112: Explorer files show Git status badges with staging details on hover. Added, modified, and deleted line runs appear beside the line numbers; context lines are not highlighted. Clicking a marker opens its selected chunk in a popup owned by that document window. Comparisons include staged and unsaved edits against the last commit, without changing the source file or Git index.
- Each document window owns its scans and selected chunk. Results carry the workspace, path, and buffer revision, so late results cannot replace another document's hunks. Background jobs repaint their originating viewport. Tests cover two different buffers of the same file alongside another repository, wrapped lines, clipping, deletion boundaries, and actual editor gutter clicks in both populated and empty documents.
- Regression correction: changing the document revision now closes the selected chunk before a new Git scan, so a popup cannot display stale content after a file switch or edit. View > Git also reveals a previously closed Explorer.

- The document status row now reports Git line totals as + added, ~ modified, and – deleted from the current buffer diff. Deleted runs retain their old-line count even though their editor range is an empty boundary, and the summary has a descriptive hover tooltip.
- Item 26: The combined Git/theme capture reproduced missing glyphs. GPU readback confirmed pixels missing from the uploaded font atlas. Immediate viewport rendering now resynchronizes the current atlas after font-cache changes and repeated parent layout passes. Ordinary frames keep incremental uploads, and existing text layouts are preserved. Three regressions cover stale parent uploads, font-cache replacement, and steady-state uploads. This reproducible transition is fixed; the older deferred report remains open for any unrelated cause. No renderer dependency fork or diagnostic code remains.
- Verification: Formatting and strict Clippy pass. All 607 repository tests pass, with 2 environment-dependent tests ignored; all 8 packaging tests pass. An initial gallery-manifest check overlapped gallery regeneration; the complete suite passed after capture finished. Six fresh release viewport captures of the Git page, editor, and selected chunk were inspected in Latte and Mocha, including the formerly failing transition. The complete 68-image gallery was regenerated in one app session, validated, and representative fresh images inspected. Existing repository index contents were preserved.

43. correctness audit (2026-09-13 code review)

- Item 113: Native asynchronous file dialogs remember their originating viewport, so completing a dialog after another window becomes current wakes the correct window. Imported files preserve their source permissions, including executable scripts, while retaining collision protection.
- Both Git diff consumers share deterministic unified-output arguments. Custom diff markers and suppressed blank context lines no longer shift editor change markers. Intent-to-add files now receive an Added badge and use an empty baseline when absent from the last commit.
- Background Git decorations obtain branch and file status in one status command without fetching history or taking optional index-refresh locks. The Git window also reuses its initial snapshot for a plain refresh instead of immediately repeating the same scan.
- Switching files or workspaces detaches obsolete read-only scans, allowing the new file to scan immediately. Typing in the same file keeps one worker and debounces the next request. Detached workers can finish their bounded operation, but cannot publish a result or error into the new document.
- Verification: Six new regression tests cover the fixes, including actual temporary Git repositories, permission preservation, viewport wake routing, and blocked-worker transitions. Formatting and strict Clippy pass; all 613 repository tests and all 8 packaging tests pass, with 2 environment-dependent repository tests ignored. These changes have deterministic behavioral coverage; no new visual verification is claimed for this audit.

44. refactoring implementation (2026-09-13)

- Items 114–118 record the next product priorities: crash recovery, Git chunk actions, workspace replacement, shared status/performance work, and PDF polish.
- Item 119: Settings, editor rendering, and native child views now have focused modules under `src/app/`; tests have their own module. Window orchestration stays in `src/windowing.rs`. `app.rs` is 14,792 lines, down from roughly 22,800 including tests. Runtime helpers remain candidates for future extraction.
- Item 120: Implemented the R1–R8 boundaries in `refactor.typ`, including a headless core crate, sealed edit/snapshot/save APIs, exclusive workflow phases, coordinated multiwindow close, bounded compiler/asset queues, retained mutation completions, an exit barrier for active background operations, preview provenance, typed coordinates, context-owned palettes, scoped workspace targets, and explicit write durability outcomes.

- The new Unicode tests found and fixed an LSP edit panic at the empty final line after a newline. Worker disconnection is now terminal rather than an indefinite loading state; PDF link extraction has a bounded, reaped process.
- Native verification reproduced a light/dark-to-new-child rendering abort. The viewport adapter now waits for the appearance-change frame before creating a new child, while retaining existing children. Capture batches also own and restore their original fixture independently of scene-specific document edits.
- Verification: Formatting, strict Clippy, all 643 workspace tests, and all 8 xtask tests pass; 2 environment-dependent workspace tests remain ignored. The complete 68-image gallery was regenerated in one app session and validated. Four fresh font, settings, save-modal, and restored-main viewports were inspected, along with representative gallery images. Native bounds traces are retained under `.tiptotyp/screenshots/refactor-review/native-geometry.log`; composed native child-window placement was not visually verified. Both Typst documents compile to private review outputs without overwriting existing PDFs.

45. correctness audit (2026-09-13 Git and document-state regressions)

- Item 130: View > Git now opens the Explorer panel when it was closed, and explicitly reveals the Git section once even if a persisted collapsing state had left it hidden.
- Item 112: Selected Git chunks are discarded whenever the owning document revision or path changes. A stale chunk popup therefore cannot remain visible after editing or switching files, while each window keeps its own selection.
- The Git Explorer section is visible by default in normal document windows; View > Git still toggles it when more file-tree space is needed. Its body starts with repository actions; the workspace path remains in the parent window header rather than being repeated inside the panel.
- Workspace/file refreshes queue a Git status scan automatically, including updates after saves, external reloads, and the periodic filesystem check.
- Git editor decorations are refreshed by document, filesystem, and Git operation events; an unchanged editor does not schedule periodic scans or repaint the interface just to rediscover the same snapshot.
- Verification: Formatting, strict Clippy, all 587 application tests (2 environment-dependent tests ignored), and all 8 xtask tests pass. A release capture attempt for `git-window` could not start a native viewport in this environment because macOS LaunchServices/HIServices reported an invalid XPC connection and then timed out; no visual pass is claimed from that attempt.

46. completed workspace, lifecycle, and sticky context issues (2026-09-14)

- ☒ Item 131: Command+, toggles Settings closed when the Settings window is already open.
- ☒ Item 132: On macOS, closing every document window keeps tiptotyp running as an application that can create or open another window.
- ☒ Item 133: Workspace history is canonicalized and deduplicated before it is saved. The workspace selector always exposes the newest 20 entries and supports removing an entry from its context menu.
- ☒ Item 134: Changing the workspace invalidates and refreshes every workspace-scoped model, including Contents, so no panel can retain paths or index data from the previous workspace.
- ☒ Item 135: Sticky context excludes single-line `#let`, `#set`, and `#show` code while retaining single-line section and subsection headings.

-  Item 136: Sticky context recognizes every multiline code construct, including multiline raw/code strings.
-  Item 137: A sticky row is pushed upward by its ending boundary as the source scrolls, matching VS Code instead of disappearing on one frame.
- Settings uses one toggle path for the native menu and keyboard shortcut. Workspace changes synchronously clear the previous project index before scheduling the file tree, index, Git decorations, compiler, and Tinymist state for the new root.
- Recent roots are normalized, deduplicated, and capped at 20 both when they enter history and when settings are saved. Removal is propagated to every document window before their histories are merged, preventing a stale window from restoring a removed root. The selector's right-click menu is covered by a semantic interaction test.
- Interactive macOS launches cancel an ordinary root-window close and hide the root after the document close workflow accepts it. Explicit Quit retains the coordinated multiwindow close path, and activating the Dock icon reveals the retained root window again.
- Sticky syntax rows now carry an explicit ending line. Multiline rules, calls, blocks, containers, equations, and raw strings participate; single-line code does not. Rows with a common ending line move as one header group.
- Verification: formatting and strict Clippy pass; all 596 application tests pass with 2 environment-dependent tests ignored, along with all integration, core, documentation, and 8 xtask tests. The release binary builds. A fresh sticky-context capture could not start its native viewport because macOS LaunchServices/HIServices returned an invalid XPC connection and the app timed out, so no new visual pass is claimed.

47. correctness audit (2026-09-14 first workspace switch)

-  Item 140: Opening a remembered document during the first workspace-root switch invalidates the previous root's Explorer snapshot. Snapshot reuse now requires both the loaded document and the cached tree to belong to the active root, so the path header, file tree, Git, Contents, tags, and other indexed sections refresh in the same transition.

48. correctness audit (2026-09-14 sticky context motion)

-  Item 141: Sticky rows with the same ending boundary slide away as one cohort. A later row ending on its own is clipped beneath the preceding sticky rows instead of painting over them. The overlay background, bottom border, gutter divider, and shadow use the moving visible bottom and leave with the text.
- Verification: formatting, strict Clippy, all 599 application tests, and all 8 xtask tests pass. The release capture attempt could not create a native viewport because macOS LaunchServices/HIServices returned an invalid XPC connection, so no fresh visual pass is claimed.

49. correctness audit (2026-09-14 compact Git marker spacing)

-  Item 142: Keep the Git hunk marker immediately beside the line-number column. Its lane must add one fixed width regardless of the document's line count instead of leaving a larger gap as line numbers gain digits.
-  Item 143: Remove redundant hover tooltips from Contents rows; the visible heading and line number already provide the useful navigation context.
- The gutter uses the measured width of the widest rendered line number. The marker fills a fixed six-point lane from the editor's left edge, while its eight-point hit target overlaps the noninteractive

number area. Formatting, strict Clippy, all 599 application tests (2 environment-dependent tests ignored), all integration, core, and documentation tests, all 8 xtask tests, and the release build pass. A fresh `git-editor` capture could not start because macOS HIServices rejected its XPC connection, so no new visual pass is claimed from this environment.

50. editor performance (2026-09-14 open Git subpanel)

-  Item 144: Keep an open Git subpanel from slowing the code editor. Build controls only for visible changed-file rows, prepare status totals in the Git worker, and retain the selected diff's colored text layout across editor frames. Invalidate that layout when its text, style, fonts, or scale changes.
- The regression fixture previously built all 10,000 changed-file rows each frame; it now builds at most ten while typing and scrolling. A scrolled row's action still targets its own path. Summary tests cover staged, partially staged, untracked, and private files; a 4,000-line diff reuses its layout until content or rendering inputs change.
- Verification: formatting, strict Clippy, 600 passing application tests (2 environment-dependent tests ignored), integration, core, documentation, architecture, and all 8 xtask tests pass. The release binary builds.

51. Git subpanel cleanup (2026-09-14)

-  Item 145: Replace each file's Stage/Unstage and Diff/Staged diff pairs with two buttons that follow its staging state. Keep each column's width stable across state changes, abbreviate to first letters in narrow panels, and retain full accessible names and hover descriptions. Remove the diff chooser instructions and redundant Commit subtitle.
- Verification: state transitions and fixed column widths pass at narrow and wide panel sizes, along with all required checks and the release build. Inspected a fresh Git viewport capture under `.tiptotyp/screenshots/agent-review` to confirm the two aligned actions and removal of the redundant text.

52. Explorer organization (2026-09-14)

- Item 131: Tags (`<label>`) and references (`@label`) have separate, independently collapsible panels. The syntax index separates them before rendering, with independent search results and navigation to each occurrence's file and line.
- Item 132: Settings → Explorer panel order provides up/down controls for every panel and a reset action. Order is saved with shared application preferences and applies to every document window. Git defaults directly below Files. A validated order contains every panel exactly once. Section identities retain their collapse and scroll state, and resize weights stay with each panel; dividers resize the next open panel in the displayed order.
- Verification: semantic tests cover reordering, reset, button alignment, collapse and body identity, independent tag/reference search and navigation, and resizing after a reorder. Persistence and invalid-order tests pass. All 608 application tests pass (2 environment-dependent tests ignored), along with formatting, strict Clippy, supporting suites, and all 8 xtask tests. Regenerated and validated all 68 gallery images, and inspected the updated Explorer layout in Catpuccin Latte and Mocha. The release build passes; a fresh `git-editor` capture under `.tiptotyp/screenshots/agent-review` confirms Git below Files and the separate Tags and References headers.

53. Window identity and matching delimiters (2026-09-14)

- Item 134: Every `ttt` logo opens a native color picker owned by its window. Changes update the logo immediately, with contrasting text, a fixed button size, and Reset/Done controls. Escape returns focus to the owner; switching windows dismisses the picker without stealing focus. Colors remain local to each document or utility window for the session and never enter shared application settings.

- Item 135: The caret highlights both ends of a Typst syntax delimiter pair, including nested code/content brackets, math delimiters, dollar signs, quotes, labels, emphasis, and raw fences. Comments, escaped characters, and literal contents cannot create false pairs. Results are cached by document identity, revision, and caret; ordinary typing skips delimiter parsing. The highlights reuse the editor galley and follow individual glyph advances at soft wraps, without rebuilding text layout on cursor movement.
- Item 132 follow-up: A wide Settings row could push the reorder controls beyond the window edge. Their layout now uses the visible width, with a regression test that precedes them with deliberately oversized content.
- Verification: all 617 application tests pass (2 environment-dependent tests ignored), together with formatting, strict Clippy, supporting test suites, and all 8 xtask tests. Fresh window-color and delimiter-match framebuffers in Catpuccin Latte and Mocha were captured under `.tiptotyp/screenshots/agent-review` and inspected, along with the colored logo in its parent window. The release build passes. Regenerated and validated all 68 gallery PNGs and inspected a fresh Settings capture confirming the reorder controls remain visible.

54. Automatic pairing and rainbow brackets (2026-09-14)

- Item 136: Auto-close delimiters defaults on and can be disabled in Settings. Typing an opener inserts its closer with the caret between them; Backspace removes an empty pair, and typing the existing closer moves over it. Typst syntax distinguishes quotes, dollars, labels, raw fences, and emphasis from operators and literal text. Brackets also pair in prose. Comments, escapes, strings, raw payloads, pasted batches, and IME composition remain literal. Native TextEdit operations retain event order, Unicode cursor positions, selections, and atomic document undo/redo. The pairing rules are stateless, so undo or switching documents cannot leave stale generated-pair positions.
- Item 137: Rainbow brackets default on, with independent nesting cycles for parentheses, square brackets, braces, and mixed math pairs. Settings offers Spectrum, Forest, Sunset, and Orchid palettes for each family, with live light/dark color samples and an independent enable switch. Preferences save and propagate to every document window. Colors preserve syntax decorations and exact byte positions, and are cached with the syntax layout rather than recalculated when the caret moves. Typing over a closer does not invalidate completions or schedule document work.
- Verification: all 628 application tests pass (2 environment-dependent tests ignored), together with all 31 core unit tests, supporting suites, strict Clippy, formatting, and all 8 xtask tests. Regression coverage includes Unicode and same-frame event order, nested/symmetric delimiters, literal exclusions, paste/IME handling, disabled pairing, character limits, undo/redo, window isolation, no-op closer movement, independent color cycles, exact syntax-layout mapping, palette cache invalidation, persistence, and semantic Settings controls. The release build passes. Regenerated and validated all 68 gallery PNGs. Fresh rainbow-brackets and bracket-settings captures in Catpuccin Latte and Mocha were inspected under `.tiptotyp/screenshots/agent-review`, including all four nesting cycles, literal exclusions, palette samples, and fixed Settings scroll positions.

55. Settings window compression (2026-09-15)

- Item 142: Reduced the default Settings window width from 620 px to 500 px, reduced font selector widths, and kept the 360 px minimum so controls wrap cleanly in narrower windows. Bracket-family labels now use `()`, `[]`, `{}`, and `mixed` to avoid unnecessary horizontal space.
- Verification: focused Settings, rainbow-label, geometry, formatting, and strict Clippy checks pass. The deterministic Settings screenshot could not be completed because macOS LaunchServices/Services was unavailable in the test environment.

56. Settings wrapping and color adjustments (2026-09-15)

- Item 143: The auto-save delay label, slider, and numeric value share one non-wrapping control. Each toolchain status chip also stays intact, moving to the next line when it cannot fit. The layout measures the full group before placing it, including the chip's frame margins.
- Item 144: Removed the extra transform explanation. Both theme pickers now offer every built-in theme, with matching light/dark themes listed first and the other set below a separator. Imported themes stay in the chosen slot, and opposite-mode choices survive saving and reloading preferences.
- Item 145: Appearance includes Luminosity, Brightness, Contrast, and Saturation controls and Reset colors. Luminosity adjusts midtones while retaining black and white; brightness shifts all channels, contrast changes their separation, and saturation adjusts color intensity. Adjustments apply to both appearance slots after inversion and hue rotation, preserve alpha, and update UI and syntax colors together from the original theme. They save with preferences and propagate across windows without reloading fonts.
- Verification: all 640 application tests pass (2 environment-dependent tests ignored), together with supporting suites, formatting, strict Clippy, and all 8 xtask tests. Semantic tests cover wrapping at 320–700 points, opposite-mode selection and ordering, and editing/resetting color controls; persistence, bounded transforms, and UI/syntax consistency are also covered. Regenerated and validated all 68 gallery PNGs. Inspected eight fresh Settings viewport captures under `.tiptotyp/screenshots/agent-review` in Catpuccin Latte and Mocha: the default-width window, 360-point editor and status layouts, and both theme pickers. The delay control and status chips remain intact, and the color controls fit in two compact rows at the default width.

57. Compact Settings hints and unified color controls (2026-09-15)

- Item 146: Settings hints use measured, wrapped plain text with compact padding, no minimum card size, and scrolling only for long content. Cards resize when the hovered control changes and stay inside the current window. Removed search-section hints, repeated preview-jump explanations, basic brightness/contrast definitions, and status hints that repeat the chip. Tool paths only show a local hint when elided or clipped. Retained useful details such as diagnostics, imported theme paths, reorder arrow actions, and the distinct meaning of luminosity and zero saturation.
- Item 147: Invert colors, Hue shift, Luminosity, Brightness, Contrast, and Saturation share one Color adjustments section. Reset colors restores all six controls together. Labels and sliders remain grouped when wrapping; preferences, transform order, and screenshot override isolation are unchanged.
- Verification: regression tests cover compact card sizing, long-to-short content changes, window-edge placement, redundant hint suppression, grouped controls, and resetting inversion/hue independently. All 644 application tests pass (2 environment-dependent tests ignored), together with supporting suites, formatting, strict Clippy, and all 8 xtask tests. Regenerated and validated all 68 gallery PNGs. Inspected six fresh Settings captures under `.tiptotyp/screenshots/agent-review` in Catpuccin Latte and Mocha: the default 500-point window, 360-point color controls, and the compact luminosity hint. The tooltip fits its single sentence, with no reserved empty space, and each slider stays with its label.

58. Maintainability and recovery follow-up (2026-09-15)

- Item 148: Recovery now uses a headless, generation-owned state machine with caller-supplied time. Failures one through four schedule a one-second retry; failure five selects fallback. Preview-start failures count even when the LSP is still alive. Duplicate shutdown events do not consume retries, stale readiness cannot recover a failed attempt, and success resets the budget.
- Item 149: Explorer visibility/width/query state has a dedicated owner. Settings rendering borrows presentation data and emits explicit application actions; it cannot reach document or worker state. Tooltip rendering, timing, geometry, and viewport caches are isolated from EditorApp.

PreviewController owns failure classification and recovery presentation. Narrow architecture checks protect the new boundaries; further application extraction is still possible and is not hidden by moving methods between files.

- Item 150: QaSession owns fixture buffers, temporary fonts, and scene/batch setup. Removed the screenshot-only Git child window and its viewport; the replacement `git-panel` scene captures the actual Explorer subpanel in `main`. No old scene-name alias remains. The fixture restores a visible Explorer width after child-window captures. Git polling runs in the app update loop, not the renderer, so rendering supplied state cannot replace it with a live scan. The real-panel capture exposed a narrow-width overlap; change counts now sit above staging actions, with regression assertions for their separation.
- Item 151: Missing settings use defaults normally; malformed settings now produce a startup notice and a status-log entry. Before replacing them, saving preserves the exact rejected input under `tiptotyp.settings.rejected`; later ordinary saves keep that record.
- Item 152: Added read-only GitHub Actions checks for Linux, macOS, and Windows, plus a separate macOS job for pinned Typst/Tinymist and Poppler integration tests. Explicit real-tool runs fail if Tinymist is missing instead of silently reporting a skipped test as successful.
- Verification: formatting, strict Clippy, the full application/core/supporting suites, and all 8 xtask tests pass. The application suite has 648 passing tests and 2 normally ignored real-tool tests; both real-tool tests also pass when explicitly run with the pinned Typst/Tinymist binaries and installed Poppler. State, protocol-event, Settings action, settings-recovery, and layout regressions cover the changed behavior. Regenerated the complete 68-image gallery in one app session and validated every PNG. Inspected fresh Settings, compact tooltip, and real Explorer Git framebuffer captures under `.tiptotyp/screenshots/agent-review` in Catppuccin Latte and Mocha, including Git on initial launch and after returning from Settings. The Git status and staging rows are separated and the Explorer width is restored. These captures do not claim native-child desktop composition coverage. The hosted CI matrix has not run yet; it will run after the workflow is pushed.
- Boundary rationale: `docs/architecture/0005-focused-ui-and-recovery.md`.

59. Settings idle performance (2026-09-15)

- Item 153: The shared child-window host sent `SetTheme` on every frame. `egui` schedules a repaint for every viewport command, even when the value is unchanged. Immediate child viewports also redraw their parent, so leaving Settings open continuously repainted both windows and exercised the native OpenGL surfaces. Native appearance is now synchronized only on creation, theme changes, and reopening a closed viewport. Embedded windows do not send native theme commands to their parent.
- Settings now emits a preference update only when the edited values differ from the current or already-pending values. Idle renders no longer allocate and dispatch an unchanged update; changing back to a live value still cancels the corresponding pending edit.
- Measurement: release builds, the same Catppuccin Latte settings-window scene and `docs/ui-snapshots/theme-fixture.typ`, no interaction, six seconds of warmup followed by a four-second macOS sample run at 1 ms intervals. The final ps CPU reading fell from 57.5% to 0.3%; CPU time accumulated between the surrounding readings fell from 2.85 s to 0.14 s. The latter interval includes sample processing. These are local measurements, not a cross-machine performance guarantee. The after sample places 3277 of 3420 main-thread samples in the event wait; the before sample instead places 3042 of 3152 in the run-loop observer driving rendering.
- Verification: regression tests first reproduced the idle repaint loop and redundant preference actions, then passed with the fixes. Coverage includes native theme changes, close/reopen, idle pending preferences, and cancelling a pending edit. Formatting, strict Clippy, all 649 non-ignored application tests, supporting/core suites, and all 8 xtask tests pass (2 real-tool tests remain intentionally ignored in the standard suite). Inspected the fresh before and after Settings viewport PNGs under `.tiptotyp/screenshots/settings-performance-before` and `.tiptotyp/screenshots/settings-`

performance-after; layout and appearance are unchanged. No native geometry or maintained screenshot contract changed.

60. Profiling foundation (2026-09-15)

- Item 2: Added an optimized, symbolized profiling build and cargo `xtask profile`. The runner preserves frame pointers and supports Settings, main editor, Find/Replace, font-picker, and large-source workloads. macOS uses native CPU samples by default; Linux can opt into perf; all platforms can select wall-time summaries without a CPU sampler.
- Runs own disposable source copies, a project-discovery boundary, a Git discovery ceiling, and fresh app-state storage. They neither inherit the surrounding checkout's repository work nor read or overwrite ordinary app preferences. Warmup starts after the initial viewport capture. Unique output directories retain logs, source/binary/tool hashes, revision/build metadata, summaries, samples, and the viewport PNG. Watchdogs handle startup, sampler, and exit failures without terminating an existing user app.
- Timing scopes cover shell/editor/Settings/Explorer UI, highlighting and search calls versus rebuilds, theme loading, compile-result handling, and worker jobs. Aggregation is bounded and opt-in, with no per-frame file writes. Percentiles are labelled histogram upper bounds; inclusive wall time is not CPU time. Normal builds compile out the hooks, including the capture-state query.
- Added cargo `bench -p tiptotyp-core --bench document` for shared snapshots, no-op edits and edit/undo at roughly 16 KiB, 256 KiB and 1 MiB. One local optimized smoke run measured snapshots at about 6 ns across all three sizes, no-op edits from 0.58 to 34.9 microseconds, and edit/undo cycles from 0.97 to 120.8 microseconds. These expose scaling for later investigation, not portable thresholds or end-to-end UI latency guarantees.
- docs/performance.md documents commands, interpretation, platform limits, reproducible comparisons, and fixture overhead. AGENTS.md now requires ongoing performance review, comparable before/after evidence for sensitive changes, and deterministic regression tests for performance bugs. CI checks both feature configurations without flaky timing thresholds. Item 117 remains open for individual large-project optimizations; item 2 establishes the tools and working practice, not a claim that every hot path is already optimized.
- Verification: formatting, strict Clippy, normal and profiling-feature full test suites, all 13 xtask tests, and the headless benchmark pass. There are 650 passing application tests normally and 655 with profiling enabled; both retain the same 2 intentionally ignored real-tool tests. Coverage includes bounded summaries, measurement boundaries, malformed timing options, fresh app state, fixture/project isolation, unique artifacts, watchdog cleanup, dormant-hook overhead, and a close timer that requires no intervening UI frame.
- Native smoke tests on macOS arm64 completed all five scenarios with complete four-second reports and no dropped scopes. The Settings run used six seconds of warmup and produced native CPU stacks with Rust symbols and source lines; the remaining scenarios used two seconds of warmup and no sampler. Inspected each fresh viewport PNG under its run's workspace/.tiptotyp/screenshots; the intended scenes and isolated files are present. This is viewport evidence, not native desktop composition proof. The final Settings artifacts are in .tiptotyp/profiles/1789482145995-89752-settings-0; the other final runs are 1789482990253-90879-main-0, 1789483000868-91180-large-0, 1789483011263-91473-find-0, and 1789483021699-90878-fonts-0 in the same profiles directory. Native profiling has not been verified on Linux/Windows, and the hosted CI matrix has not run yet.
- An earlier native run exposed an unreliable UI-scheduled exit in an idle window. A one-shot sleeping timer now wakes the native event loop and queues close independently; its regression test and subsequent native runs pass. The failed run was retained, and its owned process group was confirmed empty after watchdog cleanup. No existing user app was terminated.

61. resolved in the 2026-09-15 hover performance pass

- Item 154: Wheel/trackpad input dismisses the owning viewport's hover. Source offset changes also cover scrollbar and keyboard scrolling. Hovers remain disarmed under a stationary pointer after scrolling, so newly exposed tokens cannot immediately reopen them. Moving away from the source-to-card route or leaving the popup dismisses it; movement toward/into the card, native pointer handoff, keyboard focus, and scrolling inside the card remain supported.
- Shared text and asset hover windows now use deferred native viewports. Popup scrolling/painting no longer forces an editor paint and nested OpenGL buffer swap. Parent notifications are limited to pointer ownership/focus/ dismissal changes and link actions; stale child callbacks cannot replace another hover's state. Deferred surfaces retain the appearance-change barrier without the immediate-view font-atlas repair/upload.
- Server hover requests wait until the hover delay expires (keyboard requests remain immediate). Removed hover fading and its obsolete Settings control/key. Hover payloads and highlighted jobs are shared; cache lookups no longer clone the entire job collection. Parsed Markdown and measured block heights are retained, and offscreen blocks skip highlighting/widget construction during scrolling. Width, style, font, and content changes invalidate geometry.
- Long responses initially render at most 600 Unicode characters. Entering or keyboard-focusing the popup automatically reveals the entire response, without a Read more button, pagination, or viewport resize. This defers display work, not the server's response transfer. A first full layout still scales with document size; no claim is made that arbitrarily large hovers load for free.
- Added a native hover profiling scenario and a repeatable optimized headless probe, documented in docs/performance.md. Three final probe runs used the same 121-byte function hover and synthetic 17,470-byte/100-section Markdown. Median warm redraw means: short full 0.010 ms; long preview 0.029 ms; long full without offscreen culling 2.468 ms; long full with culling 0.085 ms (about 29x less CPU-side UI work). The full document's first layout was 8.6–9.8 ms across these runs. This probe excludes GPU/native swaps, server latency, and actual input-to-frame latency; timing numbers are evidence, not CI thresholds.
- Native optimized comparison: six-second warmup and six-second sample windows with the same isolated function-tooltip scene. Baseline artifacts: `.tiptotyp/profiles/1789483491719-94517-hover-0`; final artifacts: `.tiptotyp/profiles/1789487076673-12617-hover-0`. Mean editor pass fell from 10.01 ms to 0.411 ms, and the nested tooltip buffer-swap stack disappeared. Final popup content needed no repaint during the measurement window. Parent pass counts differed (57 versus 264), so this is not a whole-app FPS or idle CPU speedup claim. Earlier/intermediate samples remain alongside these runs.
- Verification: default/profiling strict Clippy and full tests, both formatting checks, and 13 xtask tests passed. There are 660 passing application tests normally and 665 with profiling enabled, plus the existing two ignored tests and passing support/core/integration suites. New regressions cover request gating, scroll suppression, movement direction, automatic expansion without controls, shared caches, culling geometry/invalidation, and independent child rendering/notifications.
- Regenerated and validated all 68 maintained gallery PNGs in one session; inspected gallery contact sheets and fresh function/asset viewport captures. Additional fresh captures under `.tiptotyp/screenshots/agent-review` are `1789486715589-0001-diagnostic-function-tooltip.png` and `1789486715854-0002-asset-hover-asset-preview.png`. The native geometry trace included `ui.preview.bounds` with available/egui bounds `(881.6,30.0) - (1672.0,957.1)`, native `(793.4,27.0) - (1504.8,861.4)`, zoom 0.900, egui scale 1.800 and native scale 2.000. Viewport screenshots do not establish composed desktop/WebKit geometry; that composition was not visually verified.

62. resolved in the 2026-09-15 semantic hover targeting pass

- Item 155: Reproduced the general targeting failure before changing production code: the character-only word scanner merged a math base, subscript operator, and attached identifier, then requested hover at the base rather than at the hovered identifier. Hyphenated math expressions had the same problem. Replaced that scanner with revision-cached Typst syntax-leaf targeting in `EditorDerivedData`. Valid code identifiers retain underscores/hyphens; math operators separate their operands. There are no symbol-name exceptions.
- Adjacent-leaf fallback handles egui's insertion position after a final glyph, including before spaces, superscripts, subscripts, fractions, and delimiters. The existing pointer-versus-token rectangle check still limits hit testing. Plain-text/string word hovers remain bounded to their syntax leaf. Unicode scalar-to-byte mappings use the existing source index, and unchanged document/ cursor queries reuse their result without reparsing or scanning the prefix.
- Regression coverage exercises the reported expressions, renamed symbols, mixed attachment/operator/field-access cases, valid code names, Unicode prefixes, every character position plus the trailing boundary, and cache invalidation after edits. A new opt-in real-Tinymist test also passed: all six requested positions across both reported lines and a renamed math expression returned nonempty hover content. Run it with `cargo test real_tinymist_hover_targets -- --ignored --nocapture`.
- Verification: formatting, normal/profiling strict Clippy and full suites, and all 13 xtask tests passed. Application suites: 662 normal and 667 profiling-feature passes, with three opt-in tests excluded from default runs; the new real-server test was run explicitly and passed. No popup layout, rendering, fade, or scroll-dismissal rules changed, so no new screenshot or unrelated native performance benchmark was needed for this targeting fix.

63. Settings repaint isolation (2026-09-15)

- Item 156: Settings still used an immediate native viewport, coupling every child interaction to editor rendering and nested OpenGL surface switches and swaps. It now uses the shared deferred child host with owned presentation/UI state, not a shared lock around `EditorApp`. Search, scrolling, picker navigation, hints, and font-preview completion stay local. Changed preferences, app actions, close, focus ownership, and global shortcuts notify the owner.
- Input changes explicitly invalidate the child, while unrelated editor frames do not. Font catalogs are copied only on revision/reopen/QA-scene changes, not pointer frames. Rapid preference edits coalesce, and a field-wise merge preserves concurrent preferences and document history. Native text-edit menu commands are delivered inside the next child pass, not injected into stale input; text keys and global shortcuts are consumed/forwarded once. Native menu callbacks explicitly wake the root command dispatcher.
- The first deferred-window profile exposed another problem: native preview creation intentionally waits for document-window focus, but the waiting message kept its spinner running indefinitely behind Settings. It produced 624 editor passes in six seconds. That focus-wait state is now static, with explicit activation guidance. Real loading still animates and the safeguard against stealing focus is unchanged. The profiler now retains at most 64 root repaint-request file/line counters to distinguish spinners from normal delayed polling. Free-form reasons and per-frame disk writes are excluded.
- Measurement: macOS 14.6.1, Apple silicon, optimized profiling build with frame pointers, isolated small document, Catpuccin Latte, 500 by 560 point Settings at 2x, six seconds warmup and six seconds idle sampling at 1 ms. Baseline `.tiptotyp/profiles/1789494126293-23204-settings-0` and final `.tiptotyp/profiles/1789500659316-34754-settings-0` retain metadata, hashes, summaries, and CPU samples. Editor-pass mean fell from 6.804 ms to 0.574 ms (149.70 to 14.34 ms total); Settings paints fell from 22 to 4. Per-paint Settings rendering remains approximately 1.3–1.5 ms: the main improvement is eliminating coupled work, not making every widget dramatically cheaper. Supplementary process CPU time advanced 0.17 s before versus 0.07 s after between sampler-

bracketing readings, which include sample processing. These are local idle/inclusive-wall-time measurements, not a scroll-FPS guarantee.

- Verification: 668 ordinary and 674 profiling-feature application tests pass (three opt-in tests ignored), plus all supporting/core/integration suites, 13 xtask tests, formatting, and both strict Clippy configurations. New tests protect independent parent/child wheel frames, static focus waiting, native search editing, one-shot shortcuts, close/reopen, cache invalidation, bounded repaint metadata, edit coalescing and concurrent-change preservation.
- Inspected five fresh final framebuffers under `.tiptotyp/screenshots/settings-performance-review`, beginning with `1789500841350-0001-settings-settings-window.png` and ending with `1789500841804-0005-settings-settings-font-picker.png`: light/dark Settings, narrow colors/status, and a loaded font sample. An earlier batch timed out at the font scene; scene-aware catalog invalidation fixed it and the complete rerun succeeded. No native bounds or maintained gallery layout changed; these are individual viewport checks, not composed desktop verification.

64. Architecture performance verification and protocol extraction (2026-09-17)

- Items 2, 207: matched optimized baseline comparisons found missing-import indexing regressed from 0.64 ms to 1.01 ms. A bounded, per-analysis parent normalization cache restores 0.64 ms without retaining stale filesystem state. Regression tests protect shared-parent lookup reuse and the 256-entry bound.
- Main, Settings, four-window, intact indexing, and LSP framing measurements showed no other material regression. See `docs/architecture-performance.md` for metadata, retained evidence, sampled Settings outliers, and limitations. These measurements do not establish GPU costs or long-running leak behavior.
- Item 202: extracted pure feature DTOs and codecs into `src/tinymist/protocol.rs`, retaining the public sidecar interface and wire fixtures. Borrowed notification tests protect against document copies; architecture tests reject UI, process, and thread dependencies in the module. No runtime scheduling or ownership changes were introduced. Native timings precede this pure extraction; fake-server and both real-Tinymist tests pass.
- Verification: formatting, normal and profiling-feature Clippy and full test suites, plus all 13 xtask tests and xtask formatting pass.

65. Architecture correctness audit (2026-09-18)

- Item 224: audited the refactored document, save, Tinymist, preview/PDF, project-index, workspace, capability and Git boundaries for stale identity, cancellation, rejected admission, cross-window ownership, process cleanup, bounded residency and malformed external input.
- Project-index admission is now transactional: an over-budget replacement leaves the previous queued or active request usable. Workspace subscriptions carry a generation, count unique window owners, reject out-of-root events and report both watcher-construction and watch-registration failures.
- Tinymist hover, completion and formatting acceptance now includes the complete document key, so reopening the same URI at the same LSP version cannot admit a prior document's reply. Explicit URI closure clears preview ownership. A Save As receipt that finishes after its tab is parked now rebinds that tab's workspace and language-service path without changing the active tab; renaming a parked preview tab restarts the designated preview.
- PDF metadata inspection is cancellable, timed out, reaped and bounded to 4 MiB per output stream. Incomplete Poppler page batches are rejected rather than shifted onto the wrong page numbers. Visible pages are admitted after speculative neighbours and evicted same-batch textures are dropped promptly. Raster capability reporting now requires both `pdfinfo` and `pdftoppm`.

- Git hunk ranges use checked arithmetic and must match the parsed body before an edit is constructed. Malformed external diff text therefore returns an error instead of panicking or applying a mis-sized range.
- Resource impact: the new checks run only on requests, results or filesystem events. They add no repaint loop, polling worker or steady-state document copy; PDF changes reduce transient over-budget retention. Verification passed strict formatting and Clippy, 1,045 normal tests with 16 opt-in tests ignored, and all 13 xtask tests. These are behavioral fixes, so no framebuffer or native bounds capture was required.

66. Tooltip handoff and source-jump focus regression (2026-09-18)

Item 226 supersedes the completion claims below: the first pass did not resolve the user's native interaction failures. Lifecycle cleanup still closed retained semantic popup viewports after leaving the token. Non-hovered controls erased another control's shared timer. Window activation did not restore the native first responder from WebKit. Asset hover also reparsed the full document on each pointer frame. These paths are now corrected, with regressions for actual lifecycle cleanup, shared timer ownership and one syntax rebuild across 100 pointer positions. Native end-to-end interaction and stall timings have not been measured in this follow-up; automated results alone do not establish them.

- Item 225: the popup handoff route now starts at the pointer position captured when the card opens and expands to both corners of the card's facing edge. The independent direction-based dismissal check first excludes this complete safe triangle, eliminating the race where one policy retained the popup while the other dismissed it. Entering the native child still expands and scrolls the full content; leaving the child or moving away outside the route dismisses.
- Hover timing no longer treats a frame gap over 180 ms as proof that the cursor left a widget. Current egui hover state resets the timer on a real observed exit, while delayed or event-driven frames retain the original deadline. This adds no animation, polling loop or per-frame allocation.
- A queued source selection now asks the document viewport to reclaim native focus before focusing TextEdit. This returns Option+Left/Right and other macOS editor key handling after a Tynymist preview click instead of leaving those events owned by the preview WebView.
- Verification: all 28 focused tooltip tests, both hover-timing regressions and the source-navigation focus regression pass; strict formatting and Clippy, 1,047 normal tests with 16 opt-in tests ignored, all 13 xtask tests and a `todo.typ` compile pass. The interaction is protected by deterministic geometry/state tests; no visual styling or native-view bounds changed, so no framebuffer capture was needed.